



POLITECNICO
MILANO 1863

**SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE**

Multi-Fidelity MCMC methods for efficient parameter estimation

APSC PROJECT

MATHEMATICAL ENGINEERING - INGEGNERIA MATEMATICA

Giacomo Mossinelli, 10622142

Advisor:

Andrea Manzoni

Co-advisor:

Paolo Conti

Academic year:

2023/2024

Abstract: The need to make the best possible use of one's computational resources becomes increasingly important as the complexity and size of the systems to be studied increases. For this reason, multi-fidelity and multilevel methods that exploit data from different sources in order to optimise performance are a very active field of research. This can be seen in the strictly numerical field with multi-grid methods, but also in the Bayesian field with multi-level Monte Carlo methods. In this project, we will focus on the study and implementation of certain multi-fidelity (MF) structures for the construction of neural networks, which are often used to replace purely numerical models. Furthermore, employing these MF surrogates as forward models, we will focus on their use to investigate the functionality of several python libraries and MCMC techniques to estimate model parameters from observations. The aim is to create a unified (as optimised as possible) workflow, allowing both forward models to go from input to output, and inversion techniques to estimate, given the output, the corresponding uncertain input.

Key-words: Multifidelity Neural Network Regression, Monte Carlo Markov Chain methods, Bayesian Inverse Problems, Partial Differential Equations, Parameter Estimation

1. Introduction

Scientific computing has become an ubiquitous tool across a multitude of disciplines, driving advancements in fields ranging from physics and engineering to biology and economics. At its core, scientific computing involves the application of computational methods and mathematical modeling to simulate and solve complex physical systems that are otherwise intractable through purely analytical means. These simulations allow researchers to investigate phenomena using computational experiments, providing valuable insights and predictions that guide experimental investigations and inform decision-making processes. The accuracy and reliability of these models are paramount, as they directly impact the quality of predictions and the success of subsequent applications. However, the effectiveness of scientific computing is intrinsically tied to the quality of data and the precision of the models employed. Accurate data is crucial for constructing models that can faithfully represent the underlying physical processes. Yet, obtaining high-fidelity data often comes at a significant computational cost, particularly when dealing with high-dimensional systems or complex, multi-scale phenomena. The computational effort required to achieve the desired level of precision can be prohibitively expensive, both in terms of time and resources. This creates a critical bottleneck, as the need for precision must be balanced against the practical constraints of computational feasibility.

Historically, model development in scientific computing relied heavily on deterministic approaches, where parameters were fixed and models were calibrated based on empirical data or expert knowledge. These classical

methods, while foundational, often struggled with uncertainty quantification and the integration of disparate sources of data. As computational power grew, the complexity of models expanded as well, but this advancement also intensified the challenges related to computational cost and data acquisition.

In recent years, the advent of machine learning (ML) has revolutionized scientific computing, leading to the emergence of the new scientific machine learning (SciML) paradigm [5]. This interdisciplinary field leverages the power of modern ML techniques to enhance and accelerate traditional computational models. SciML integrates data-driven methods with physical models, enabling more accurate predictions while simultaneously reducing computational costs. Among the most promising approaches in SciML are multi-fidelity neural networks. With this approach, we are able to exploit data achieved by many sources denoted by different precision levels. In the most common case scenario, together with the high-fidelity (HF) data, achieved with the most accurate (and, most of the time, expensive) solver, we have low-fidelity (LF) information which can be obtained quicker and with a smaller effort but that are describing the phenomena less accurately. For example, LF data may be obtained by a noisy measurement, by a cheaper numerical method or by a coarser discretization in a finite element method framework. An accurately tuned interaction between different precision levels gives the possibility to build less expensive models with the same precision as a more usual neural network trained on a high-fidelity dataset or a strongest model without an increasing of the computational needs. Because of the potential, relevance and success of this approach, it represents a very active research area [15–17, 26, 34, 39, 40, 44]. In the report, many possible approaches will be presented and tested in relation to many test cases in this context.

A further important element, when one wants to simulate and model a phenomenon, is – provided a sufficient number of observations – the ability of inferring unknown parameters of a model from observed data, a process that is inherently uncertain. Bayesian methods address this uncertainty by treating the parameters as random variables equipped with suitable probability distributions, incorporating both prior information and the likelihood of observed data. However, one of the most challenging aspects of Bayesian inference [53] is the computation of the posterior distribution, which is often analytically intractable.

In the context of complex scientific models, accurately estimating parameters under uncertainty is a critical challenge. Bayesian inverse problems [53] offer a rigorous framework for addressing this challenge by providing a structured approach to parameter estimation that accounts for uncertainty in a principled manner. This framework is particularly powerful when dealing with inverse problems, where the goal is to infer unknown parameters from observed data, often in scenarios where the parameter space is vast, and the models involved are highly non-linear.

Monte Carlo Markov Chain (MCMC) techniques [47] are indispensable in this context. MCMC methods generate samples from a probability distribution by constructing a Markov chain that converges to the desired distribution as its equilibrium state. These samples enable the approximation of the posterior distribution of the parameters, which is crucial for Bayesian inference. The strength of MCMC lies in its ability to navigate high-dimensional spaces and complex models, making it a powerful tool for solving Bayesian inverse problems.

To further enhance the efficiency and accuracy of parameter estimation, multi-fidelity methods are employed. These methods combine high-fidelity data, which is typically expensive to obtain, with more readily available low-fidelity data. This combination allows for a balanced approach that optimizes both accuracy and computational efficiency. When integrated with the Bayesian framework, multi-fidelity methods significantly improve the robustness and reliability of parameter estimation in scientific modeling, enabling more effective decision-making under uncertainty.

In this report, we explore the intersection of multi-fidelity neural networks and Bayesian inverse problems, highlighting their potential to improve parameter estimation in scientific computing. We begin by examining the fundamental principles of these approaches, respectively in sections 2 and 3, followed by an analysis of their application to real-world problems. In section 4, the implementation part is introduced: in general terms, the structures created for the operation of the library are discussed, as well as the interactions between them, the management of the libraries used and the interlocking of code sections relating to backward and forward models. In addition, a couple of preliminary examples through which the behaviour of MFNNs and the MCMC methods on these surrogate models are discussed in detail. Section ?? is devoted to an in-depth study of more up-to-date methods than the typical random-walk metropolis-hastings to be applied in the Bayesian inverse problems field. These innovative methods are then introduced into the library and used on example cases, in which both the effectiveness of a multi-fidelity approach and its interaction with multilevel versions of MCMC can be appreciated. Finally, in Section 6, in addition to an introduction to LSTMs, two cases of nontrivial complexity are considered to introduce the interaction between multifidelity models, ROMs, and LSTMs and then, finally improve and implement the library to be able to individuate input parameters even in this particularly elaborate case study. Through this exploration, we aim to demonstrate how these cutting-edge techniques can improve model accuracy. The results obtained, in fact, highlight the interactions that exist

between forward and backward surrogate models, showing their potentials and interactions. Moreover, the construction of an adaptable and easily extensible library that unites these two worlds provides a first tool to move smoothly in this field.

2. Artificial Neural Networks (ANNs)

Artificial Neural Networks (ANNs) (Figure 1) are computational models inspired by the human brain, designed to approximate functions through a layered structure [50, 56]. An ANN typically consists of an input layer, multiple hidden layers, and an output layer. Each layer is composed of neurons, where each neuron transforms its inputs through a weighted sum followed by a non-linear activation function.

For a network with L hidden layers, the output of the i -th neuron in the l -th layer, before activation, is given by:

$$z_i^l(\mathbf{x}) = b_i^l + \sum_{j=1}^{M_{l-1}} W_{ij}^l x_j^{l-1}(\mathbf{x}),$$

where:

- $\mathbf{x}$ is the input vector,
- W_{ij}^l are the weights connecting the j -th neuron in layer $l-1$ to the i -th neuron in layer l ,
- b_i^l is the bias of the i -th neuron in layer l ,
- M_{l-1} is the number of neurons in layer $l-1$.

After applying an activation function ϕ , the output of the neuron is:

$$x_i^l(\mathbf{x}) = \phi(z_i^l(\mathbf{x})).$$

The process continues through all layers until the output layer, where the final network output $\mathbf{y}$ is obtained:

$$y_i(\mathbf{x}) = z_i^{L+1}(\mathbf{x}) = b_i^{L+1} + \sum_{j=1}^{M_L} W_{ij}^{L+1} x_j^L(\mathbf{x}),$$

where $\mathbf{y}$ is a vector representing the predicted outputs. Thus, the ANN can be viewed as a parameterized function:

$$\mathbf{f}^{\text{NN}}(\mathbf{x}; \mathbf{W}, \mathbf{b}),$$

where $\mathbf{W}$ and $\mathbf{b}$ represent all the weights and biases across the network. The network learns these parameters through training on input-output data pairs, enabling it to generalize to new inputs.

The training of the model is typically conducted by minimizing a cost function. In our cases, it will be expressed as:

$$(\mathbf{W}, \mathbf{b}) = \arg \min_{\mathbf{W}, \mathbf{b}} \left\{ \frac{1}{N} \sum_{k=1}^N \left\| \mathbf{y}^{(k)} - \mathbf{f}^{\text{NN}}(\mathbf{x}^{(k)}; \mathbf{W}, \mathbf{b}) \right\|_2^2 + \lambda \|\mathbf{W}\|_2^2 \right\}, \quad (1)$$

where the first part represents the mean squared error (MSE), and the second term is a regularization term where $\lambda \geq 0$ is a penalty coefficient.

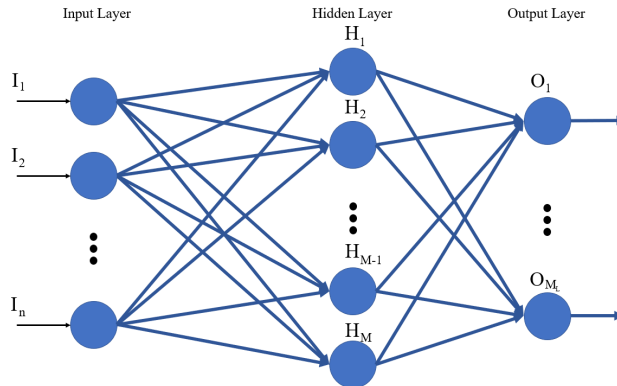


Figure 1: General structure of a Fully connected Neural Network.

To accurately describe a specific ANN, many hyperparameters must be considered and described. In fact, the choice of hyperparameters, significantly impacts its performance. The implementation of this process will be explained more accurately in section 4.1.4, but it is worth to have an idea of the considered elements in our cases.

- **Number of nodes.** One key aspect of hyperparameter optimization (HPO) involves determining the appropriate number of nodes in each layer, as this influences the model’s capacity to learn and generalize. Too few nodes may lead to underfitting, where the model fails to capture the underlying patterns in the data, while too many can cause overfitting, where the model becomes excessively complex and sensitive to noise.
- **L^2 regularization penalty.** Regularization techniques, such as L^2 weight regularization, are also essential to prevent overfitting [18]. L^2 -regularization adds a penalty term to the loss function, which discourages large weights in the network and thus promotes simpler, more generalizable models.
- **Learning rate.** The learning rate is another critical hyperparameter, governing the step size at which the model updates its weights during training. A learning rate that is too high can cause the model to converge too quickly to a suboptimal solution, while a rate that is too low can lead to slow convergence and prolonged training times.
- **Kernel initializer.** The choice of kernel initializer determines how the initial weights of the network are set before training begins. Some examples are the uniform initializer or the Glorot uniform initializer [24], often favored due to its ability to maintain a balance in the variance of activations across layers, facilitating smoother gradient flow during backpropagation.
- **Optimizers.** Optimizers, like Adam and Adamax [31, 35], are algorithms used to adjust the weights based on the gradients computed during backpropagation. Adam is widely used due to its adaptive learning rate and momentum, which generally lead to faster convergence. Adamax, a variant of Adam, is particularly robust in handling large gradients and has shown to perform well in specific cases.

To build and train our ANNs, the library `Keras`, an open-source neural network library written in Python, has been used [25, 38, 41].

2.1. Data preprocessing phase

In the context of machine learning in general, great importance is attributed to the preliminary data analysis phase, a fundamental component for learning. In this context, certain manipulation techniques can be introduced to ensure that the network (or, more generally, the model in object) is able to extract useful information from it. Below is a brief description of the most relevant methods used in the construction of some example cases.

2.1.1 Normalization

Normalization is a technique used to scale the features of input data so that they have a specific range or distribution. This is crucial for machine learning algorithms that are sensitive to the scale of data, such as gradient descent-based algorithms and those involving distance calculations (e.g., K-Nearest Neighbors, SVMs) [3]. In this report, whenever you introduce a normalization of data, we will be referring to a min-max normalization, of the form:

$$x'_j = \frac{x_j - \min(x_j)}{\max(x_j) - \min(x_j)},$$

where x_j is the original feature value and x'_j is the normalized feature value.

2.1.2 Transformation

Transformation in machine learning refers to the application of a mathematical function to the features or target variables to achieve certain properties. Transformations are often applied to linearize relationships, stabilize variance, or make the data more suitable for modeling [9]. Some common transformations are, for example, polynomial transformation or kernel transformation [32] or any transformation that can be useful to catch some properties of the initial data.

2.1.3 Data augmentation

Data Augmentation is a technique used in machine learning to increase the diversity of training data without actually collecting new data (Figure 2)[11, 37, 61]. This is particularly useful when the available dataset is small or imbalanced. By generating additional training examples, data augmentation helps prevent overfitting, improve model generalization, and make the model more robust to variations in the input data. In particular, in our case we will mainly make use of data-augmentation through the introduction of noise [42]. The idea is to make the method more robust to small changes in the input that might occur in real-world scenarios. In general, we make use of additive Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma^2)$ such that:

$$x' = x + \epsilon,$$

where, x is the original data point and x' is the new value produced with noise introduction.

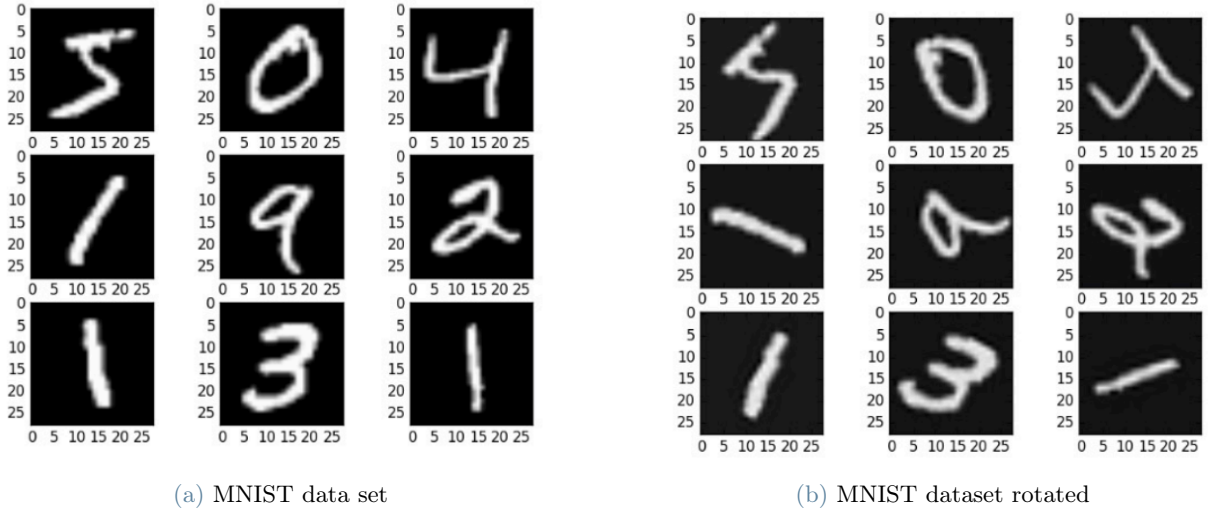


Figure 2: Example of data augmentation. On the left, the dataset is considered in its original form, on the right it is represented after a rotation.

2.2. Regularization techniques

2.2.1 Dropout

Dropout is a powerful regularization technique designed to prevent overfitting by addressing the issue of co-adaptation among neurons [7, 19, 52, 64]. During training, dropout temporarily removes a random subset of units (along with their connections) from the network (Figure 3).

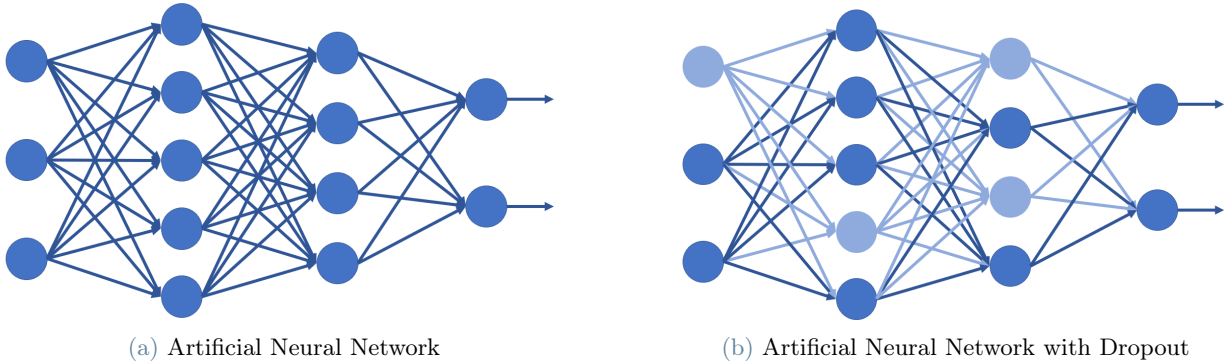


Figure 3: Comparison of an artificial Neural Network with and without dropout. In the right figure, it is possible to notice that some neurons are not activated.

Specifically, for each neuron, a binary mask m_i is sampled from a Bernoulli distribution with a probability p of keeping the neuron active. If the neuron is dropped (i.e., $m_i = 0$), its output is set to zero; otherwise, the output is retained and scaled by $\frac{1}{p}$ during training to maintain the expected sum of inputs unchanged. This random deactivation forces the network to distribute the learned representations across many neurons, reducing the reliance on any single neuron and enhancing the network's ability to generalize. At inference time, dropout is not applied; instead, the full network is used, with each weight scaled by p to account for the absence of dropout. This technique has been shown to significantly reduce overfitting in various tasks, particularly in scenarios with complex models and limited training data.

The combined application of normalization, dropout and data augmentation forms a comprehensive approach to enhancing the performance of deep neural networks. Normalization stabilizes and accelerates training, dropout prevents overfitting by promoting redundancy and robustness in learned representations, and data augmentation enriches the training set, fostering better generalization. Together, these techniques contribute to the creation

of more powerful and reliable neural network models capable of tackling complex and varied tasks. However, in order to evaluate network performance objectively and quantitatively, some estimates must be introduced to allow us to compare the performance of different models.

2.3. Model evaluation

To assess the quality of the surrogate and compare it with analogous architectures to determine the better case, we use mean squared error (also used as the loss function during model training) and the coefficient of determination R^2 [57], in addition to a qualitative graphical analysis comparing the approximation with the expected relationship.

2.3.1 Mean Squared Error (MSE)

The mean squared error (MSE) is defined as the average squared difference between the true output values y_i and the prediction $\hat{y}_i$ produced by the model:

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (2)$$

with N denoting the number of data points. A lower MSE indicates a closer fit to the true data, signifying higher predictive accuracy. In any case, attention must be paid to the fact that the MSE is only an indicator of the quality of the model, but is not by itself able to describe performance completely. For example, it is easy to see how the presence of outliers can greatly impair the value of this indicator, even though they are exceptions in an overall potentially good model. Consequently, it is useful to couple this estimate with other metrics.

2.3.2 Coefficient of Determination R^2

The R^2 statistic complements MSE by quantifying the proportion of variance in the dependent variable that is explained by the surrogate model:

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2} \quad (3)$$

with N denoting the number of data, $\bar{y} = \frac{1}{N} \sum_{i=1}^N y_i$ and $\hat{y}_i$ the prediction produced by the model. If R^2 is denoted by value of 1 it indicates perfect prediction, while a value of 0 suggests that the model does not correctly explain the model. A high R^2 value is indicative of the model's ability to capture underlying patterns in the data, thus reflecting its utility in approximating the true function.

2.4. Multifidelity Neural Network

2.4.1 Description

In recent years, the development of multi-fidelity neural networks (MFNNs) has garnered significant attention in the field of machine learning, particularly in the context of multi-fidelity regression. The concept of multi-fidelity regression revolves around the integration of data of varying fidelity levels to enhance the accuracy of predictive models trained on a single dataset.

In the context of multi-fidelity surrogate modeling, the selection and effective correlation between high-fidelity (HF) and low-fidelity (LF) models are pivotal for balancing computational efficiency with predictive accuracy. Let us consider $f_{LF}(x)$ and $f_{HF}(x)$ represent, respectively, the low- and the high-fidelity approximation of the quantity of interest $f(x)$ [26]. To exploit the benefits of both models, we should have that $f_{HF}(y)$ delivers a close approximation to the true quantity (involving fine discretization or comprehensive representation of the underlying physical processes, ensuring minimal error) and that $f_{LF}(y)$ offers computational expediency and retains a meaningful correlation with $f_{HF}(y)$ even if less precise. The LF component may arrive from simplified physics, coarser discretization, or other forms of model reduction. Despite its lower accuracy, the LF model can potentially capture the essential trends and relationships present in $f(y)$. Through a judicious selection of LF models that maintain a strong correlation with HF counterparts, one can significantly reduce computational costs without sacrificing the predictive power essential for accurate simulations or analyses. This strategy not only optimizes the use of available data but also enhances the overall effectiveness of MFNNs in surrogate modeling applications.

2.4.2 Possible architectures

One of the primary approaches proposed for the application of neural networks to multi-fidelity regression is based on the construction of distinct NNs for each fidelity level. We denote this class of MFNNs as *multilevel* architectures. Two networks are utilized to construct bi-fidelity models that approximate the scalar target HF function $f_{HF}(x)$ incorporating information from the LF data.

To formalize this framework, let N_{HF} and N_{LF} denote the number of HF and LF samples, respectively. The training set for the LF function is then represented as $\mathcal{T}_{LF} = \{(x_i, y_{LF,i})\}_{i=1}^{N_{LF}}$ with $y_{LF} = f_{LF}(x)$ and $\mathcal{T}_{HF} = \{(x_i, y_{HF,i})\}_{i=1}^{N_{HF}}$ with $y_{HF} = f_{HF}(x)$. The approximations are respectively $\hat{f}_{LF}(x)$ and $\hat{f}_{HF}(x)$. In particular, for this first phase we will focus on networks with a structure similar to that described in [26].

The multilevel architectures are then:

- "2-step" model.** A "2-step" architecture is made of a sequence of 2 NNs: the first one is trained on $\mathcal{T}_{LF}$ to learn the LF function $f_{LF}(x)$. Then, using the first NN (called NN_{LF}), it is possible to predict the values of the LF function at the training inputs of the HF data, $\{f_{LF}^{NN}(x_{HF}^i) : 1 \leq i \leq N_{HF}\}$. Finally, the second network NN_{HF} approximates the HF function $y_{HF} = F(x, y_{LF})$ based on the input $\{(x_{HF}^i, f_{LF}^{NN}(x_{HF}^i)) : 1 \leq i \leq N_{HF}\}$ and the available HF output data $y_{HF} = y_{HF}(x_{HF}) = \{y_{HF}^i : 1 \leq i \leq N_{HF}\}$. In general, NN_{HF} is a smaller NN consisting of few hidden layers.
- "3-step" model.** With respect to the "2-step" model, an additional level of fidelity, generated by a third network NN_{lin} . NN_{lin} tries to simplify the modeling of the correlation function $y_{HF} = F(x, y_{LF})$. No nonlinear activation function is used in NN_{lin} , such that NN_{lin} is responsible for capturing the linear correlations between the HF and LF data sets. Let $y_{lin} = f_{lin}^{NN}(x_{HF}, f_{LF}^{NN}(x_{HF}))$ denote the set of outputs of NN_{lin} at the HF input locations x_{HF} . The result will be used as part of the inputs for the third and final network NN_{HF} . NN_{HF} approximates the correlation function $y_{HF} = F'(x, y_{LF}, y_{lin})$, y_{lin} being the output of NN_{lin} . NN_{HF} is again a simpler NN with respect to the LF counterpart. The idea is that the 3-step model should only present an advantage over the 2-step model when there exist a sort of correlation that can be captured by the introduced network.

These structures, in general, show performance that can be improved, depending on the problem under consideration and the relationship existing between different degrees of input fidelity, by introducing, in series or in parallel, networks that capture different features of the case under study.

3. Parameters estimation through Monte Carlo Markov Chain

In several scientific and engineering fields, ranging from optics, geophysics, and acoustics to medical imaging, the need to accurately infer the underlying parameters of a model from indirect or incomplete observations is critical. These tasks are often framed as inverse problems, where the goal is to reconstruct or estimate unknown quantities that cannot be observed directly. The Bayesian framework has become increasingly vital in addressing these challenges, offering a robust probabilistic approach to quantify uncertainties in the inferred solutions.

Bayesian inverse problems are particularly significant due to their ability to incorporate prior knowledge and to provide a full probabilistic characterization of the solution space. Unlike classical methods that might yield a single best estimate, Bayesian approaches generate a posterior distribution over the possible solutions, allowing for more nuanced decision-making and better management of uncertainty.

However, solving Bayesian inverse problems is computationally demanding. It requires sophisticated algorithms that can efficiently explore high-dimensional parameter spaces, integrate complex models, and handle the often large-scale data involved. Advances in algorithms, particularly those based on Markov Chain Monte Carlo (MCMC), Variational Inference, and other modern techniques, have made it feasible to tackle these challenges in a computationally efficient manner. These methods allow practitioners to solve increasingly complex inverse problems, thus driving forward innovation in various applications [53].

The importance of developing and refining algorithms for Bayesian inverse problems cannot be overstated. They enable us to handle more intricate models, larger datasets, and more rigorous uncertainty quantification than was previously possible. As computational resources continue to grow, the development of these algorithms remains a key area of research, essential for the continued advancement of scientific discovery and technological innovation [30].

3.1. Bayesian Inverse Problem

Inverse problems involve inferring unknown parameters or states $\theta \in \mathbb{R}^p$ from observed data $d \in \mathbb{R}^q$, which is assumed to be related to θ through a forward model $d = \mathcal{F}(\theta) + \epsilon$ [27] [54]. Here, $\mathcal{F}(\theta)$ represents the forward model that maps the parameter space to the data space, and ϵ denotes the measurement error, typically assumed to be Gaussian, i.e., $\epsilon \sim \mathcal{N}(0, \Sigma_\epsilon)$, where Σ_ϵ is the covariance matrix of the noise.

In the Bayesian framework, the solution to the inverse problem is formulated as the posterior distribution $\pi(\theta|d)$, which combines prior knowledge about the parameters $\pi_{\text{pr}}(\theta)$ and the likelihood $\pi(d|\theta)$, using Bayes' theorem:

$$\pi(\theta|d) = \frac{\pi(d|\theta)\pi_{\text{pr}}(\theta)}{\pi(d)}, \quad (4)$$

where $\pi(d)$ is the evidence, a normalizing constant ensuring that the posterior distribution integrates to one. The likelihood function $\pi(d|\theta)$, reflecting the probability of observing data d given parameters θ , is typically given by:

$$\pi(d|\theta) \propto \exp\left(-\frac{1}{2}(\mathcal{F}(\theta) - d)^\top \Sigma_\epsilon^{-1}(\mathcal{F}(\theta) - d)\right). \quad (5)$$

This formulation assumes that the noise ϵ is additive and Gaussian.

Solving the inverse problem then involves exploring the posterior distribution $\pi(\theta|d)$ to estimate the parameters θ . This is commonly achieved through sampling methods such as Markov Chain Monte Carlo (MCMC), which generates samples $\{\theta^1, \theta^2, \dots, \theta^N\}$ that, after a sufficient number of iterations, are distributed according to $\pi(\theta|d)$.

In practice, the Metropolis-Hastings (MH) algorithm is widely used for MCMC sampling. Given a proposal distribution $q(\theta'|\theta^j)$, which suggests a new state θ' based on the current state θ^j , the MH algorithm accepts the proposed state θ' with a probability given by the ratio:

$$\alpha = \min\left(1, \frac{\pi(d|\theta')\pi_{\text{pr}}(\theta')q(\theta^j|\theta')}{\pi(d|\theta^j)\pi_{\text{pr}}(\theta^j)q(\theta'|\theta^j)}\right); \quad (6)$$

otherwise, the algorithm retains the current state θ^j . This approach allows for the computation of sample-based estimates of expected values with respect to the posterior distribution, such as the mean or variance of the parameter estimates, thereby providing a probabilistic solution to the inverse problem.

Despite its power, the Bayesian approach to inverse problems presents several challenges, including the selection of prior distributions, computational efficiency, and the convergence of MCMC methods. These issues necessitate careful consideration and often require advanced techniques, such as adaptive MCMC or variational inference, to ensure robust and efficient inference.

The aim of this part of the project is therefore to implement, through the support of some python libraries relating to uncertainty quantification, algorithms that enable the estimation of the starting parameters that define the behavior of a given system. This can be applied not only to the initial computational models, but also to further approximations obtained through a wealth of numerical strategies like, e.g., surrogate models, reduced order models, etc. Here, we will focus on the application with respect to models approximated through the contribution of multi-fidelity neural networks. An implementation that allows for an economic definition of both the forward model and the backward model would allow for a good understanding of the mathematical model it represents both from inputs to outputs and vice versa.

3.2. Metropolis-Hastings

The Metropolis-Hastings (MH) algorithm is a MCMC method for obtaining a collection of random samples from a distribution from which direct sampling is problematic. Through an acceptance-rejection phase, MH is able to approximate the distribution. In the context of Bayesian inverse problems, MH allows to explore the posterior distribution of the unknown parameters, from which statistical estimates and uncertainty quantifications can be used to describe the parameters themselves [12].

Algorithm 1 Metropolis-Hastings Algorithm

- 1: **Input:** Target $\pi(x)$ and proposal $q(x'|x)$ distributions, Initial state x_0 , Number of iterations N
- 2: **Output:** Sequence of states $\{x_t\}_{t=0}^N$ approximating $\pi(x)$
- 3: Initialize $x_0 = x_{\text{initial}}$
- 4: **for** $t = 1$ to N **do**
- 5: Sample $x' \sim q(x'|x_{t-1})$
- 6: Compute the acceptance probability:

$$\alpha = \min \left(1, \frac{\pi(x')q(x_{t-1}|x')}{\pi(x_{t-1})q(x'|x_{t-1})} \right)$$

- 7: Sample $u \sim \text{Uniform}(0, 1)$
 - 8: **if** $u \leq \alpha$ **then**
 - 9: Set $x_t = x'$
 - 10: **else**
 - 11: Set $x_t = x_{t-1}$
 - 12: **end if**
 - 13: **end for**
 - 14: **Return:** Sequence $\{x_t\}_{t=0}^N$
-

3.2.1 Diagnostic instruments

When applying the Metropolis-Hastings algorithm, or any Markov Chain Monte Carlo (MCMC) method, to Bayesian inverse problems, it is crucial to assess the quality of the Markov chain produced. This involves ensuring that the chain has appropriately explored the parameter space and that the samples can be used to accurately approximate the posterior distribution. Here, we describe some instruments to evaluate the quality of the simulations [48]:

- The effective sample size (ESS) represents the number of independent samples that would give you the same level of information as the correlated MCMC samples and it is defined as:

$$\text{ESS} = \frac{N}{1 + 2 \sum_{k=1}^{\infty} \rho_k} \quad (7)$$

where:

- N is the number of samples.
- ρ_k is the autocorrelation at lag k .

ESS is crucial because it helps determine the quality of the MCMC sample. A high ESS indicates that the chain provides a lot of independent information, making the sample more reliable for statistical inference. When the ESS is low, it suggests that the chain is highly autocorrelated..

- The autocorrelation at Lag k measures the correlation between the values of the sequence at time t and at time $t + k$:

$$\rho_k = \frac{\sum_{t=1}^{N-k} (x_t - \mu)(x_{t+k} - \mu)}{\sum_{t=1}^N (x_t - \mu)^2} \quad (8)$$

where:

- x_t is the value of the sequence at time t .
- μ is the mean of the sequence.

Autocorrelation helps in understanding the dependence structure within a sequence of samples, especially in MCMC. High autocorrelation means that consecutive samples are similar, reducing the efficiency of the sampling process. Understanding autocorrelation is key to diagnosing the mixing of the Markov Chains.

- Rhats (Potential Scale Reduction Factor):

$$\hat{R} = \sqrt{\frac{\hat{V}}{W}} \quad (9)$$

where:

- W is the within-chain variance

$$W = \frac{1}{M} \sum_{i=1}^M s_i^2, \quad (10)$$

with M denoting the number of chains and s_i^2 the variance within the i -th chain.

- $\hat{V}$ is the posterior variance estimate of the target distribution.

It is a diagnostic metric used to assess the convergence of MCMC chains. It compares the variance between multiple chains to the variance within each chain.

Rhat is widely used to determine if MCMC chains have converged to the target distribution. If $\hat{R}$ is close to 1, it suggests that the chains have mixed well and converged. Values significantly greater than 1 indicate that the chains have not converged, and further sampling or adjustments might be necessary.

Most of these metrics can be achieved, also with a graphical representation, by the `MCMC` and `MCMC_cuqi` classes, better introduced in section 4.2, at the end of a simulation to assess the quality of the generated Markov chains.

4. Implementation

This section will discuss the structure of the project in a more general way, thus providing an introduction to the chosen structures and an accurate explanation of the most relevant and complex steps.

First, it is worth looking at it from a more general point of view with the help of Figure 4:

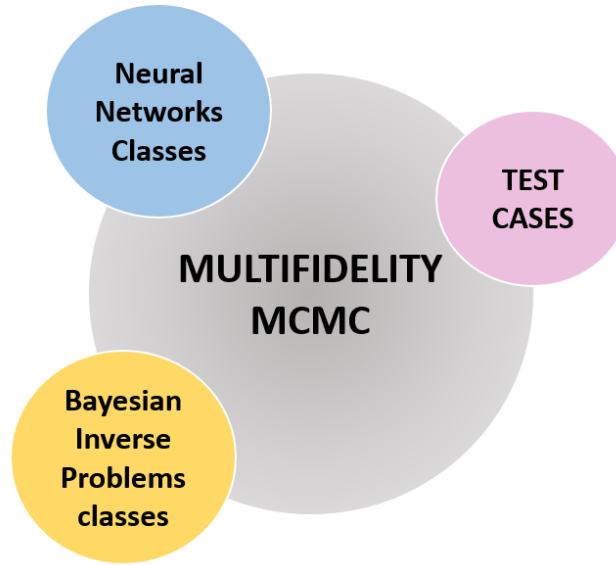


Figure 4: Division of the tasks

The idea is to create a structure as general and automatized as possible, so that it can be easily expanded to new cases and maintained. In particular, the main tasks are of two types: creation of a surrogate forward model by implementing different neural network structures and solving inverse problems through a Bayesian approach. These two macrocategories can therefore gather the ‘core functions’, in git, grouped in two different files of the ‘utils’ folder together with some very generic functions gathered in the ‘helpers_functions’ file. The ‘utils’ folder then also contains two smaller files, namely ‘MOD_helper’, which contains a small class for Model Order Reduction examples, and ‘functions_to_ray’, with some functions to be shared in parallel implementations. In addition to these, test cases are of particular importance, the structure of which is collected in the ‘source’ folder. These represent cases of varying nature and complexity, in order to show capabilities and to test the implemented functionalities. These files should fill most of the results in this report, and can be tested through the files in ‘test’. At the beginning of each chapter of this report, the name of the corresponding file or folder is given. For some examples, we opted to use a folder to group functions relating to the same test case, but with different facets, or for convenience as they have data collected by different software. In the next sections, all those elements will be explored in more in details.

4.1. Neural Network Classes

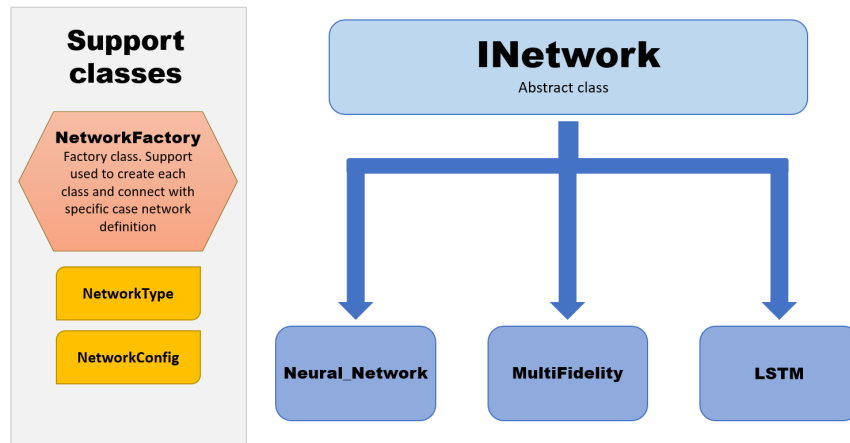


Figure 5: Main classes used to define the structure of the different networks.

For the construction of the neural networks presented in the report, the classes system represented in Figure 5 is defined and collected in `network_utils.py` file, in `utils`. For greater readability and ease of handling, they are divided into two groups, the first one, whose main character is the class `NetworkFactory`, made of classes used to manage the definition of the different NNs, and `INetwork` and its derived classes, which collects the real structure of the networks. The decision to construct a factory function and to divide the structures in this way lies in the desire to have a framework that would allow a network to be defined as easily as possible in the test files. In fact, the presence of multi-fidelity networks, particularly if they were of a multi-level type, would make the network definition cumbersome and difficult for a user to read, as one is dealing with several datasets (with different degrees of fidelity) and several networks at the same time. Instead, the structure thus created allows for greater cleanliness and immediacy. Furthermore, the division and management provided by the factory class not only makes the library easily extendable to possible new multi-fidelity structures (or, in general, to other neural networks), but also allows us to reduce code duplication as much as possible.

4.1.1 Network definition

Exploiting these classes, a network can be easily defined by the user in the following way:

```
# parameters of the Neural Network
parameters = {'lr' : 0.0255,
              'kernel_init' : 'glorot_uniform',
              'opt' : 'Adam'}

# inputs for the definition of a Single level Neural Network
definition_NN={
    "network_type": "HF",                # type of network (here, high
    "network_parameters": parameters,    # parameters to define the network
    "dataset_train": data["xhf"],         # training set
    "output_train": data["Yhf"],          # output of the training set
    "epochs_number": data["NepoHF"],     # Epochs
    "batch_size": data["Nhf"],            # batch size
    "train": True,                        # The NN is trained
    "do_HPO": False,                     # Do not Hyperparameter Optimization,
                                         # If parameters with better estimated
                                         # performance are found, they will
                                         # substitute the actual parameters
    "verbose": False                     # reduce unnecessary printed output
}

# initialization of the neural network
config = NetworkConfig(**definition_NN)
```

```
modelHF=NetworkFactory.build_network(config,getModel)
```

where *getModel* is a function containing the structure of the Keras network, receiving number of inputs, outputs, type of network name and a parameters dictionary. In our cases, *getModel* is in the class associated to the related test case:

```
class testcase:
    #...
    @staticmethod
    def getModel(params: Dict[str, Any], num_inputs: int, name: str, num_outputs: int
    ) -> tf.keras.models.Model:
        """
        Create and return a compiled Keras model based on the specified parameters
        and model type.

        Parameters:
        - params: Dictionary containing model configuration parameters, such as
            'nodes', 'l2weight', 'kernel_init', 'alpha', and 'opt'.
        - num_inputs: Integer specifying the number of input features.
        - name: String indicating the model type ('HF', 'LF', 'Single', 'Hflin', '
            Hfper', 'GP', 'Inter').
        - num_outputs: Integer specifying the number of output features.

        Returns:
        - A compiled Keras model tailored to the specified model type.
        """

        # Define the input layer with the given number of input features
        inputs = Input(shape=(num_inputs,))

        # High-Fidelity (HF) Model
        if name == 'HF':
            hidden1 = Dense(params['nodes'], activation='tanh', kernel_regularizer=l2
                (params['l2weight']),
                            kernel_initializer=params['kernel_init'], name=f'{name}
                                _hidden1')(inputs)
            output = Dense(num_outputs, activation='linear', name='HF')(hidden1)
        # Low-Fidelity (LF) Model
        elif name == 'LF':
            # ...

        model = Model(inputs=inputs, outputs=output)
        opti = getOpti(params['opt'], params['lr'])
        model.compile(loss='mse', optimizer=opti, metrics=['mse'])
        return model
```

Given this definition, the creation of a multilevel network is straightforward: it is sufficient to define a list of elements for each voice of the definition, one element per network. For example, in a 2-step network we have:

```
##### 2 steps NEURAL NETWORK #####

# parameters for the LF (first) Neural Network
bestLF_params = {
    'lr': 0.0255,
    'kernel_init': 'glorot_uniform',
    'opt': 'Adam'
}

# parameters for the HF (second) Neural Network
bestHF_params = {
    'kernel_init': 'uniform',
    'l2weight': 0.00010018625799978436,
    'lr': 0.07093837044166487,
    'nodes': 74,
    'opt': 'Adamax'
}
```

```

# Collection of elements of both networks in a ordered list
params = [bestLF_params, bestHF_params]
# Epochs
N = [data["NepoLF"], data["NepoHF"]]
# Batch sizes
n = [data["Nlf"], data["Nhlf"]]
# Names of the networks, in the right order
names = ['LF', 'HF']

# Collection of the parameters for the "2-step" Neural Network.
# Here a 2 levels case is shown, but this can be generalized to any number of levels
# after providing the definition of the structure of the NNs in the support file
definition_2steps = {
    "network_type": "2step",
    "names": names,
    "network_parameters": params,
    "dataset_train": [data["xlf"], data["xhf"]],
    "output_train": [data["Ylf"], data["Yhf"]],
    "epochs_number": N,
    "batch_size": n,
    "train": True,
    "do_HPO": False,
    "verbose": False
}
# Initialization of the Neural Network
config = NetworkConfig(**definition_2steps)
final_model=NetworkFactory.build_network(config, testcase)

```

and this can be generalized to N networks.

4.1.2 Neural Network "support" classes

The most important class in this group is `Network_factory`, which centralizes the network creation process. This has the role of linking the class of the test case (more precisely, its `getModel` method) with the chosen network type and consequently dynamically create network instances based on the provided `NetworkConfig`. The most important methods are therefore

- `build_network`, which checks that the desired network is among those defined,
- `"_build_type_of_network"` methods (such as `_build_neural_network`), which call the constructor of the corresponding class in the file.

The factory pattern allows the system to create objects without specifying the exact class of the element that will be created, thus promoting flexibility and scalability.

The other classes, strictly related to the first one, are `NetworkType` which is intended to collect the existing structures preventing the use of invalid or unintended values (type safety), and the `NetworkConfig` data class, which is a centralized container that encapsulates all the necessary details required to build and train a network. Its role is to simplify and organize the configuration process by gathering all related parameters into a single, coherent structure. This structure is then passed around in the system, making it easier to manage and modify the configuration without having to handle numerous individual parameters separately.

4.1.3 INetwork class

Each type of neural network architecture considered consists of a class of its own. These classes, in addition to presenting necessary functionalities such as *prediction* and *training*, collect the parameters and characteristics of the network to be set up as well as additional functionalities depending on the structure we are analysing. To avoid repetition of functions as much as possible, the classes are collected under the umbrella of **INetwork**, designed to define a common interface for various types of network models. It is defined as an abstract class which, in addition to indicating functions that each underlying class must implement, collects the common functionalities of the derived objects, which are thus acquired by inheritance. In some of these cases, the capabilities of **INetwork** are extended by overriding.

One of the most important things to notice is the role of the `MultiFidelity` class. In order to exploit the already implemented and optimized functions for the `Neural_Network` class and to define it with a structure that externally looked as much as possible as a unique network (as shown in Section 4.1.1), the `MultiFidelity` class is basically a list of `Neural_Network` objects, whose connection is managed automatically by the class itself,

without any external needs. As a consequence, the most important functions, such as training and prediction needed redefinition to make them suitable for the list-type structure. Some of the most important common (but with different definitions) functions are:

- **training** called to train the network with the given hyperparameters. This method can be called on an already defined network or done since the beginning setting 'train'=True.
- **prediction** given an appropriate input, it gives back the output following the estimated surrogate model.
- **HPO** Hyperparameter optimization, performed with **Optuna** library. A better description is given in Section 4.1.4.
- **param_inverse** and **inverse_cuqi** functions used for the parameter estimation process to exploit the created surrogate model as a forward model.

4.1.4 Hyperparameter optimization

In the design and deployment of neural networks (NNs), the selection of hyperparameters plays a crucial role in determining the model's architecture and overall performance. Hyperparameters, such as the number of layers, learning rate, and dropout rate, must be meticulously chosen to enhance the model's ability to generalize from training data to unseen test data. Unlike model parameters, which are learned during the training process, hyperparameters are predefined and have a significant impact on both the convergence speed and accuracy of the final model.

The process of hyperparameter optimization (HPO) is inherently a complex task, often requiring iterative trials and extensive experimentation to identify the optimal settings. Traditional methods, including manual tuning, grid search, and random search [4, 20, 33], are not only time-consuming but also computationally expensive, especially as the complexity of neural networks increases. While these methods are straightforward, they often fail to discover the best-performing hyperparameters, particularly in high-dimensional spaces where the number of possible configurations is vast. Recent advancements in automated hyperparameter optimization techniques [60, 63] have introduced more efficient and systematic approaches to address these challenges. Techniques such as Bayesian optimization [23], genetic algorithms, and gradient-based optimization have emerged as powerful tools to navigate the hyperparameter space more effectively. These approaches aim to automate the selection process, reducing the need for manual intervention and enabling a more thorough exploration of potential hyperparameter values.

Optimizing hyperparameters is crucial not only for improving model performance but also for enhancing the efficiency of the training process. Since HPO, most of the time, is very time-consuming, it would be good to be able to divide the necessary work between several workers through parallelisation. A possible library that can exploit parallelization and reduce the required computational-time is **Optuna**. **Optuna** is a hyperparameter optimization framework designed to efficiently navigate the high-dimensional and often complex search spaces inherent in machine learning models. The library is particularly focused on robust parallel execution capabilities, making it a powerful tool for enhancing model performance in computationally demanding scenarios [1].

Bayesian optimization is a key strength of **Optuna**, implemented using the Tree-structured Parzen Estimator (TPE) [59]. Bayesian optimization is a sequential model-based optimization method that relies on constructing a probabilistic model of the objective function. The key idea is to use a surrogate model (in this case, TPE) to approximate the true objective function and then use this model to decide where to evaluate the objective function next. This approach is particularly useful for optimizing expensive-to-evaluate functions.

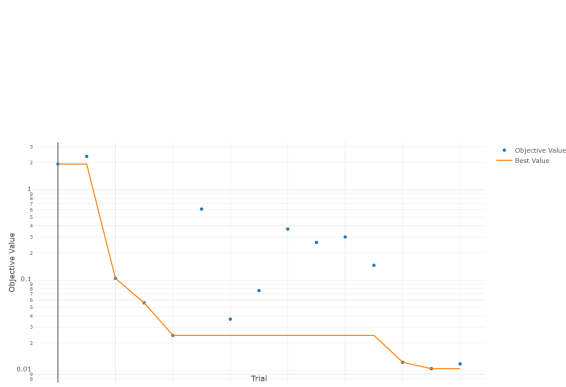
This method iteratively refines this model based on observed results, thereby focusing the search on promising regions of the hyperparameter space. Unlike traditional grid or random search methods, which can be inefficient in high-dimensional spaces, Bayesian optimization in **Optuna** allows for a more intelligent exploration that balances exploration and exploitation. The TPE algorithm in **Optuna** models the likelihood of a set of hyperparameters leading to good results. It splits the observed hyperparameters into two groups: those that yield the best results (below a certain threshold) and all others. It then estimates two probability density functions, one for each group, and selects new hyperparameters that maximize the expected improvement, focusing the search on areas likely to produce optimal results.

By focusing computational resources on more promising areas of the search space, TPE reduces the number of evaluations needed to find near-optimal hyperparameters, which is particularly advantageous when dealing with expensive-to-evaluate objective functions, such as those in deep learning or complex ensemble models. The search space can be adjusted based on previous trials' outcomes or other runtime conditions, making it particularly well-suited for complex workflows where the hyperparameter space is not known a priori or is conditional on other variables. The ability to adapt the search space during the optimization process enables the optimization of hyperparameters that are interdependent, thus allowing a more nuanced and effective search

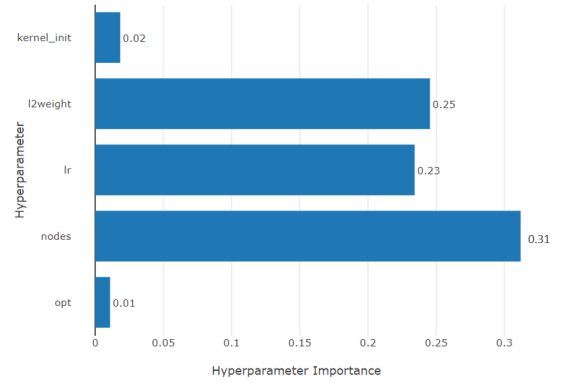
strategy.

By pruning trials that are unlikely to yield better results, **Optuna** reduces unnecessary computations, which can significantly decrease the total time required for optimization, especially in cases where each trial is computationally expensive. As previously mentioned, **Optuna**'s design includes robust support for parallel and distributed execution, allowing it to scale effectively across multiple CPUs, GPUs, or even distributed computing environments. Parallelization in **Optuna** is achieved through its integration with external frameworks like Dask or Ray, which manage the distribution of trials across available resources. The ability to parallelize trials leads to significant reductions in optimization time, especially for models with long training durations. Furthermore, **Optuna**'s asynchronous execution model minimizes idle times by immediately launching new trials as soon as resources become available, rather than waiting for all trials in a batch to complete.

Optuna's capabilities make it particularly suitable for scenarios where hyperparameter tuning is critical to model performance and where computational resources are a limiting factor. The combination of Bayesian optimization, dynamic search space definition, and parallel execution allows the library to efficiently navigate complex, high-dimensional hyperparameter spaces with a reduced computational burden. The only drawback lies in reproducibility. When **Optuna** comes into play, parallelisation does not allow, in this context, the same identical hyper-parameters to be obtained again later.



(a) Objective value decrease through the HPO process



(b) Indication of the importance of each hyperparameter for the optimization.

Figure 6: Information given by **Optuna**'s Dashboard

However, it is possible to obtain a history of the information thanks to the online Dashboard provided by the library, which gives a summary of the optimisation process by providing some insights, such as those represented in Figure 6.

4.1.5 Further details

To optimise the memory space occupied by these structures and to define an element as clean as possible, it was decided to use the dataclass decorator with the `__slots__` feature for many classes. In Python, a dataclass is a decorator that automatically generates special methods like `__init__`, `__repr__`, and `__eq__`, making it easier to create classes that are primarily used to store data. When you define a class with `__slots__`, Python does not use a dynamic dictionary (`__dict__`) to store instance attributes. Instead, it allocates a fixed amount of space for each of them, based on the `__slots__` declaration. This can lead to significant memory savings, especially if you're creating a large number of instances. Without `__slots__`, each instance carries the overhead of a dictionary for attribute storage, which can be quite large for objects with many attributes. Since `__slots__` removes the dynamic dictionary, attribute access becomes faster. Although the tested cases are not extremely heavy, this improvement fits in well with the class structure and does not lead to any kind of deterioration in terms of performance. Furthermore, it must be remembered that a multi-level network is, in fact, the union of several interacting networks. Consequently, it seems reasonable to think of a structure that can well manage the memory that will have to be used.

Another detail concerns the possibility of GPU use. In functions for which this could be advantageous, the use of a GPU is implemented. This can provide benefits, especially during the training phase, where mixed precision training is used, which helps save computational resources. Tested on the time-consuming case of training the

LSTM of Chapter 6.2, it was verified that the time required for CPU training averaged almost 1800 seconds, while on GPU it takes only 400 seconds. Google Colab was used for this type of experiments.

4.2. Bayesian Inverse Problem Classes

In order to accomplish parametric estimation through the use of MCMC, three main objects were defined inside of `bayesian_utils.py`, in `utils`: `BayesianInverseProblem_NN`, `ProposalAlgorithm` and `BayesianLibraryBase`. Starting from `BayesianInverseProblem_NN`, which is the only class seen by the user, it is possible to choose the appropriate sampling algorithm to perform Monte Carlo Markov Chain. Here an example that shows how to use `BayesianInverseProblem_NN`.

```
#### Bayesian Inverse Problem ####

# given a dataset with some observations related to the parameter we want to
# estimate through a Bayesian approach

parameters = np.array([[3.75]])          # the chosen parameter
mean_prior = np.array([10.])           # mean of the initial gaussian
cov_prior = np.diag([1])                # covariance matrix of the prior

sigma_noise_bounds = (0.25, 0.5)        # noise covariance on the observation
n_data_bounds = (30, 50)                # range of number of observations
sigma_bounds = (0.5, 1.5)               # standard deviation of the likelihood
rwmh_scaling_bounds = (0.5, 1.5)        # rwmh step
rwmh_cov = 1
rwmh_adaptive = True
iterations = 1500                        # iterations of the markov chains
burnin = 1000                           # burn-in steps
n_chains = 2                             # number of chains
algo = "MH_tiny"                         # Metropolis-Hastings of tinyDA library,
# proposal algorithm

# definition of the object for Bayesian Inversion Problem
BIP = BayesianInverseProblem_NN(algorithm_name=algo, forward_NN=final_model)

# hyperparameter optimization
BIP.hpo(inputs_HF=data["xhf"],
        domain_bounds=(0., 5.),
        mean_prior=mean_prior,
        cov_prior=cov_prior,
        output_HF=data["Yhf"],
        sigma_noise=sigma_noise_bounds,
        number_data=n_data_bounds,
        real_parameters=parameters,
        sigma=sigma_bounds,
        rwmh_scaling=rwmh_scaling_bounds,
        rwmh_covariance=rwmh_cov,
        rwmh_adaptive=rwmh_adaptive,
        iterations=iterations,
        burn_in=burnin,
        n_chains=n_chains,
        levels=1,
        force_sequential=False,           # force_sequential=False means to
# parallelize the chains' simulation in tinyDA
        initial_points_optimizer=3,
        iterations_optimizer=5           # steps of the optimizer:
# initial_points_optimizer + iterations_optimizer
    )

# running only an MCMC is analogous, with the run method, for example ...
sigma_noise = 0.15
n_data = 37
sigma = 2.5
rwmh_scaling = 2.911
rwmh_cov = np.eye(2)
rwmh_adaptive = True
```

```

iterations = 2000
burnin = 1250
n_chains = 2
algo = "MH_tiny"
set_seed()
BIP=BayesianInverseProblem_NN(algorithm_name=algo,forward_NN=final_model)

BIP.run(
    inputs_HF=data["datatest"],
    output_HF=data["modified_highfid"](data["datatest"][:,0],data["datatest"][:,1]),
    mean_prior= mean_prior,
    real_parameters=parameters,
    iterations= iterations,
    burn_in= burnin,
    rwmh_adaptive=rwmh_adaptive,
    cov_prior= cov_prior,
    n_chains= n_chains,
    domain_bounds=(0.,5.),
    sigma_noise=sigma_noise,
    number_data=n_data,
    sigma=sigma,
    rwmh_scaling=rwmh_scaling,
    force_sequential=True,
    levels=1
)
# if already used hpo with the same algorithm before, the parameters subject to
the research can be avoided (the run method, will use the best ones achieved
in the HPO)

```

In a similar way, if we use an algorithm relate to CUQIpy:

```

#### Bayesian Inverse Problem ####

# given a dataset with some observations related to the parameter we want to
estimate through a Bayesian approach

mean_prior = np.array([15])
cov_prior = np.diag([5.])

parameters = np.array([21.])

cov_noise = 0.09649 # covariance noise
n_data= 31 # number observations
iterations = 2000
burnin = 1250
number_chains = 2
proposal_sd = 0.6484
scaling = 1.
adapt=True
set_seed()
algo="MH_cuqi" # chosen algorithm. Here, for example,
               random walk metropolis hastings
BIP=BayesianInverseProblem_NN(algorithm_name=algo,forward_NN=final_model)

BIP.run(
    inputs_HF=data["datatest"],
    output_HF=data["modified_highfid"](data["datatest"][:,0],data["datatest"][:,1]),
    mean_prior= mean_prior,
    real_parameters=parameters,
    iterations= iterations,
    burn_in= burnin,
    cov_prior= cov_prior,
    sd_noise=cov_noise,
    proposal_sd= proposal_sd,

```

```

num_data=n_data,
scale= scaling,
adapt=adapt,
number_chains= number_chains,
domain_bounds=(0.,5.),

parallel=False                                # True to have parallel Markov Chains
)

```

With `BayesianInverseProblem_NN` we can call the method `hpo` that finds the optimal value of some hyperparameters for solving the inverse problem between a user-specified range, and `run`, that just performs a single simulation with the given parameters or with the optimal ones previously found with `hpo`. Those are performed with the help of the two other more complex classes.

4.2.1 BayesianLibraryBase

For parametric estimation through MCMC, two libraries, `TinyDA`[51] and `CUQIpy`[2, 6, 46], were studied. The use of both is due to the desire to compare different implementations and to enrich the range of choices available for proposal algorithms. With the aim of using both libraries without making them too cumbersome to use, it was decided to introduce a base class `BayesianLibraryBase`, which, through the support of the smaller `LibraryConnection`, allows the algorithm given to `BayesianInverseProblem_NN` to be linked to the reference library. The inversion method then has slightly different definitions (collected in `Inversion_tinyDA` and `Inversion_cuqi`) depending on whether one class associated with one library or the other is relied upon. A graphical representation of the relationship within the classes can be seen in Figure 8. The classes `MCMC` (which exploits `TinyDA`) and `MCMC_cuqi` are then used only by the class associated with the related library.

It is worth introducing the `hpo` procedure: using Bayesian Optimization, this approach is particularly effective when working with high-fidelity (HF) data, which could originate from either simulations or experimental observations.

The process begins by ensuring that all necessary parameters are in place: since not all the available algorithms are subject to the same hyperparameters, depending on the chosen one the bayesian optimization investigates only certain parameters. Moreover, an hyperparameter is investigated only if it is given in the form of tuple (with minimal and maximal values) while, if it is given as a scalar, it is not investigated.

The procedure highlights the combination of parameters that results in the smallest relative error to be best.

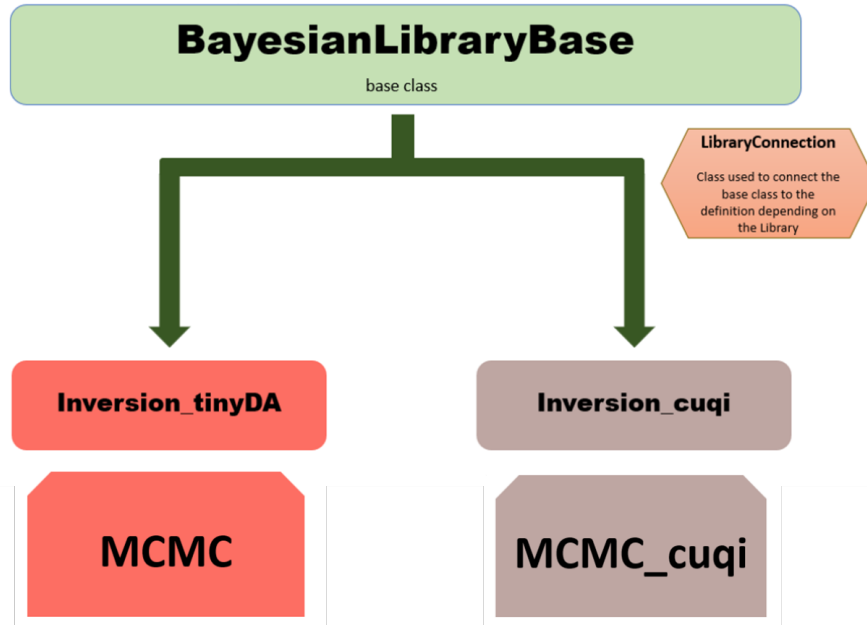


Figure 7: Classes structure for Bayesian Inverse Problem implementation

4.2.2 ProposalAlgorithm

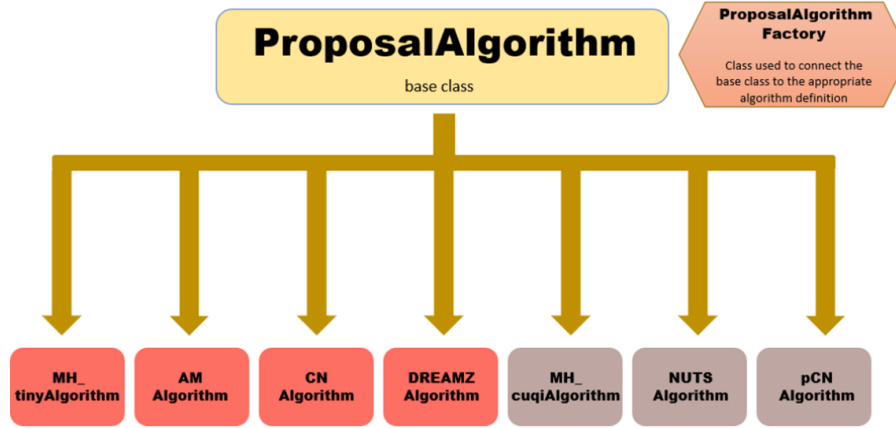


Figure 8: Classes structure for Bayesian Inverse Problem implementation

The last relevant class to be introduced is `ProposalAlgorithm` whose role is to collect each proposal algorithm. Specifically, each algorithm has the function `setup_proposal`, which defines the proposal itself, and `get_parameters`, which defines the parameters needed for the specific algorithm to be defined. This is done in such a way that, in the context of the hyperparameter search, irrelevant elements take on a placeholder value. Each algorithm is encapsulated within its own class (e.g., `MH_tinyAlgorithm`, `AMAlgorithm`, `CNAlgorithm`, etc.). This separation makes the code easier to maintain, extend, and debug since changes to one algorithm don't affect others.

4.2.3 Parallelization

Parallelization plays a crucial role in scientific computing, offering substantial benefits by enabling the concurrent execution of tasks across multiple processors or cores. This approach significantly reduces computation time and enhances efficiency, particularly for computationally intensive operations. By distributing workloads, parallelization accelerates processing, allows for the management of larger datasets, and optimizes resource utilization. Furthermore, it improves throughput and scalability, ensuring that programs can efficiently leverage modern multi-core processors and distributed systems, including clusters and cloud environments. In particular in Markov Chain Monte Carlo (MCMC) methods, parallelization is critical for improving computational efficiency, especially when dealing with high-dimensional and complex probabilistic models. As MCMC techniques often require extensive sampling to accurately approximate posterior distributions, the computational demand can be prohibitive. Since individual Markov chains are independent, they can be computed separately, making them ideal candidates for parallel execution. This distribution of sampling tasks across multiple processors significantly reduces convergence times, enabling the practical application of MCMC in large-scale problems. Two libraries for solving Bayesian Inverse Problems are being considered in the project, namely, `TinyDA` and `CUQIpy`. While the former is already equipped with a process for parallelising Markov chains, the latter lacks it. This is why, through the use of the `Ray` library, the parallelisation of these processes was introduced. `Ray` [43] is a Python-based distributed computing library that simplifies the development of scalable and parallelized applications. It provides a user-friendly API for distributing tasks across multiple cores, machines, and clusters, making it well-suited for workloads demanding significant computational resources. The core of `Ray`'s functionality lies in its remote functions (`@ray.remote`), which enable parallel execution of functions across different processes or nodes. `Ray` also leverages actors to support stateful computations, allowing for the encapsulation of state that persists across tasks—a feature critical for long-running computations and distributed services. `Ray`'s architecture is designed for efficient task scheduling, employing a distributed scheduler that dynamically allocates tasks to available resources, thereby optimizing load balancing within the cluster. The system also incorporates fault tolerance, automatically detecting and recovering from task failures to enhance the robustness of distributed applications. Moreover, `Ray` integrates seamlessly with leading machine learning frameworks such as `TensorFlow` and `PyTorch`, supporting parallelized training, hyperparameter tuning through `Ray Tune`, and scalable model serving via `Ray Serve`. In summary, `Ray` is designed to be user-friendly while offering the power to manage complex, large-scale computations with minimal overhead.

Below is an excerpt of the code (from `bayesian_utils.py`) with this type of introduction, as well as a performance measurement summarised in a graphical representation of the results obtained on the CPU (Figure 9). By studying the figure, it can be seen that for a rather large number of chains, a fair time advantage can be

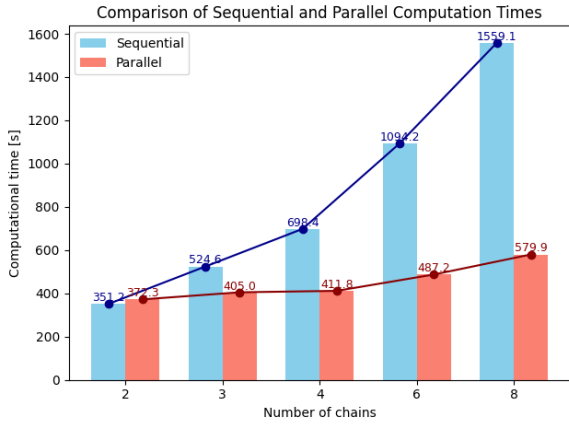
achieved, while for a few chains, performance is basically equivalent. This is mainly due to the fact that the higher the number of chains, the more the communication time is offset by the gain due to the use of more workers.

```
# class for MCMC with CUQIpy library
class MCMC_cuqi:
    ...
    def __call__(self, initial_point: np.ndarray, N: int, burn_in: int, n: int = 1,
        ...) -> Tuple[np.ndarray, Dict[str, Any]]:
        ...

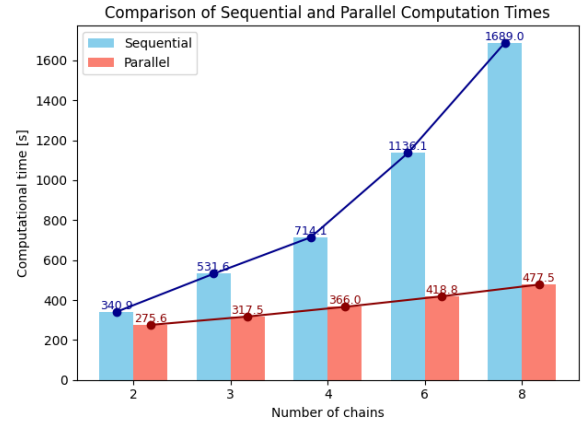
        if self.parallel: # boolean value to call for parallelization
            ray.init(ignore_reinit_error=True, logging_level=30)
            futures = [self.chain_creation_parallel.remote(N, burn_in, posterior,
                initial_point) for _ in range(n)]
            results = ray.get(futures)
            ray.shutdown()
        else:
            results = [self.chain_creation(N, burn_in, posterior, initial_point) for
                _ in range(n)]

        ...

    @ray.remote
    def chain_creation_parallel(self, N: int, burn_in: int, posterior: Any, x_init:
        np.ndarray) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
        # call to the definition of the Markov Chain
        return self.chain_creation(N, burn_in, posterior, x_init)
```



(a) Time measurement of Random Walk Metropolis Hastings chains creation with TinyDA



(b) Time measurement of Random Walk Metropolis Hastings chains creation with CUQIpy

Figure 9: Comparison of the time required to generate x Markov chains with the 2 tested libraries

4.3. helper_functions.py and MOD_helper.py

These 2 files contains additional classes in order to run some of the examples. In particular, `MOD_helper.py` contains the class `MOD`, a class with some additional functions used for examples *LSTM_burger.py* (Example IV in the report) and *LSTM_reaction_POD* (Example V in the report). `helper_functions.py` contains some classes for different purposes, such as *Clean* with a couple of functions used to create folders, *decorators*, which contains a collection of decorators, *Inversion_helper* with some additional functions for BIP execution and *Helpers_NN* with some functions to reduce the code replication in the management of neural networks.

4.4. Example I: Reaction-diffusion problem

Regression is a fundamental technique in both statistics and machine learning, serving as a primary tool for modeling relationships between variables and making predictions. In the context of machine learning, regression algorithms are extensively used to predict continuous outcomes based on input features, with methods ranging from simple linear models to complex neural networks. A key challenge in regression tasks, particularly in multi-fidelity environments, is the integration of datasets that vary in accuracy and computational cost. Multi-fidelity (MF) regression addresses this by leveraging HF data, which is often sparse and expensive to obtain, alongside LF data, which is more abundant but less accurate. The goal of MF regression is to optimize predictions by exploiting the complementary strengths of both data types, allowing for more accurate and computationally efficient models. This approach is particularly useful in scenarios where high-precision data is limited, enabling the use of LF data to approximate trends and refine predictions derived from HF data. Recent advancements have explored the application of artificial neural networks (ANNs) in MF regression, showing promise in effectively combining data of varying fidelity to achieve improved performance across a range of applications [26].

As a preliminary example, we consider a lambda-omega reaction-diffusion system:

$$\begin{aligned} u_t &= (1 - (u^2 + v^2)) u + \mu (u^2 + v^2) v + d(u_{xx} + u_{yy}), \\ v_t &= -\mu (u^2 + v^2) u + (1 - (u^2 + v^2)) v + d(v_{xx} + v_{yy}) \end{aligned} \quad (11)$$

with initial condition

$$u(x, y, 0) = v(x, y, 0) = \tanh\left(\sqrt{x^2 + y^2} \cos\left((x + iy) - \sqrt{x^2 + y^2}\right)\right), \quad (12)$$

defined over a spatial domain $(x, y) \in [0, 20]^2$ and a time span $t \in [0, T]$ for $T = 80$. The parameters μ and d regulate the reaction and diffusion behaviors of the system. Periodic boundary conditions are imposed. The system has been studied for a fixed diffusion coefficient and with reaction term $\mu \in \mathcal{P} = [0.5, 1.5]$. The aim is to show the potential of a multi-fidelity approach to model the relationship from input variables to output quantities related the solution of this problem. The possibility of having a simple but sufficiently powerful architecture to approximate output quantities of such a complex example could represent a considerable saving in computational terms compared to the typical numerical solver. Both HF and LF solution datasets are constructed by solving the PDEs (11) using the Fourier spectral method [55]. The chosen time step is $\Delta t = 0.05$.

Both datasets were defined over the entirety of the time domain through the application of uniform discretization. The difference between the two is denoted by a different discretization (more refined in the HF case). Furthermore, a further distorting element of the LF dataset with respect to its HF counterpart is denoted by the diffusion parameter, which is taken differently with respect to the HF case. The latter choice may represent a laboratory case, in which the instrumentation is not able to perfectly identify the values defining the problem, thus yielding dirtier data, while instead, the different discretization is intended to make the LF dataset simpler and more manageable from a computational point of view. More precisely, the HF dataset is generated using $n_{HF} = 100$ points in each spatial direction. For the LF dataset, we have tried different discretizations, namely $n_{LF} = \{10, 28, 46\}$. An example is reported in Figure 10.

Therefore, let us start from a case where most of the input variables are fixed. Specifically, let us only consider what happens at the final instant, at a point in space, specifically $(x, y) = (2.23, 2.23)$. This choice relies on the fact that we wanted to focus on a grid-point common to all the considered discretization levels and avoiding the centre, which may show a too simple behaviour.

Hence, the relationship we want to study at this point is the one that links the reaction parameter μ to the solution u , at a fixed time and point in space, i.e. $\mu \mapsto u(x, y, t; \mu)$. This relationship, although denoted by a single input and output, is complex to model with a DNN given its oscillatory behaviour, which differs between LF and HF cases (see Figure 19). Therefore, in order to successfully model this relationship, elements of data augmentation were introduced through the addition of data soiled with Gaussian noise and of a second feature z_2 with the transformation

$$z_2 = \left| \sin\left(5\pi \cdot \mu - \frac{5\pi}{6}\right) \right| \quad (13)$$

which tries to point out the oscillatory behaviour to the network. Finally, a normalization of the input and output data has been performed.

Computational performance could be improved but not substantially, even with these ingredients, due to the oscillatory behaviour. Then, a new additional layer was finally introduced. Inspired by [64], which provides rigorous proofs that feedforward networks with ReLU and tanh activations cannot extrapolate periodic functions – indeed, these activations either lead to linear or constant behavior in the asymptotic limit – we introduce a

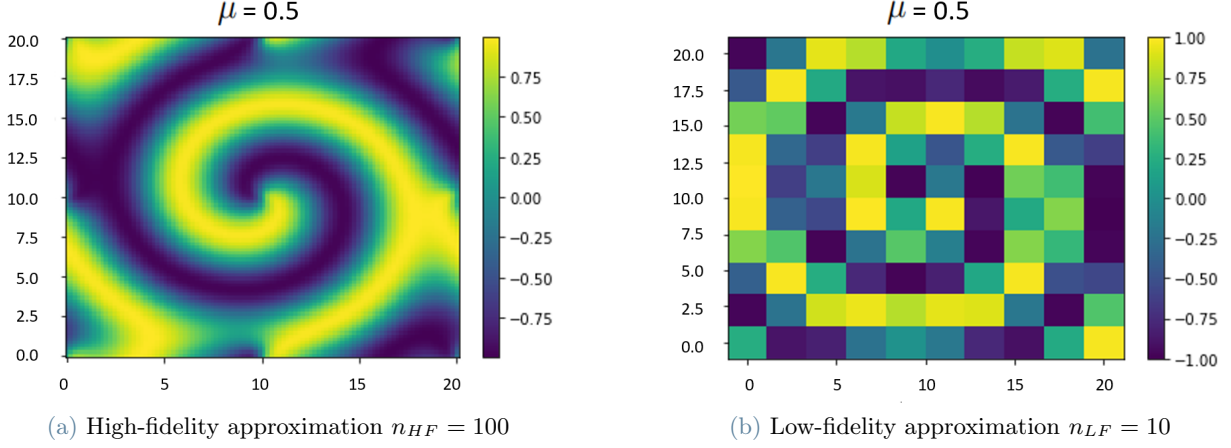


Figure 10: Solution of (11) obtained through the numerical solver for $\mu = 0.5$.

keras-like layer (called the `FourierLayer`) better able to capture such behaviour. The `FourierLayer` is a custom neural network layer designed to introduce periodicity into the feature space via sinusoidal transformations. Given an input vector $\mathbf{x} \in \mathbb{R}^d$, the layer produces an output vector $\mathbf{y} \in \mathbb{R}^d$, where each element is computed as:

$$y_i = \sin(\alpha_i x_i) + \cos(\beta_i x_i) \quad \text{for } i = 1, 2, \dots, d,$$

with α_i and β_i being learnable parameters initialized as:

$$\alpha_i, \beta_i \sim \text{Uniform} \left(-\sqrt{\frac{6}{d}}, \sqrt{\frac{6}{d}} \right).$$

The primary function of the `FourierLayer` is to embed input data into a feature space where periodic patterns are more easily captured. The sine and cosine functions, both periodic with a period of 2π , enable the layer to model functions with periodic components. The learnable parameters α_i and β_i control the frequency of these periodic functions, making the model adaptable to various periodic behaviors. The layer can be seen as performing a learned Fourier-like transformation. The `FourierLayer` is particularly useful in capturing cyclic patterns in time-series data or decompose signals into their frequency components, akin to a learned Fourier transform. Compared to standard layers, this layer adds non-convexity to the optimization landscape but may be less suited to tasks requiring non-periodic feature extraction.

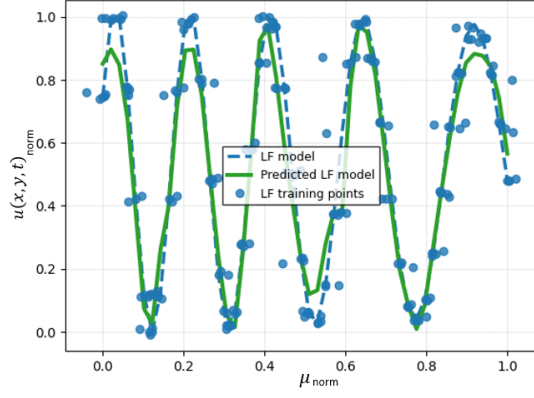
After an hyperparameter optimization, studying the number of nodes, the initializer, the learning rate, the optimizer and the l_2 -regularization weights to find optimal parameters, the "2-step" NN achieves variable results depending on the LF dataset under consideration (Table 1 and 2). This demonstrates that the first component of the multilevel network plays an important role in defining the quality of the architecture. The evolution of the performance is graphically represented in Figure 12, where the HF reference set is plotted against the prediction of the "2-step" network.

4.4.1 Comparison with Low-Fidelity and High-Fidelity models

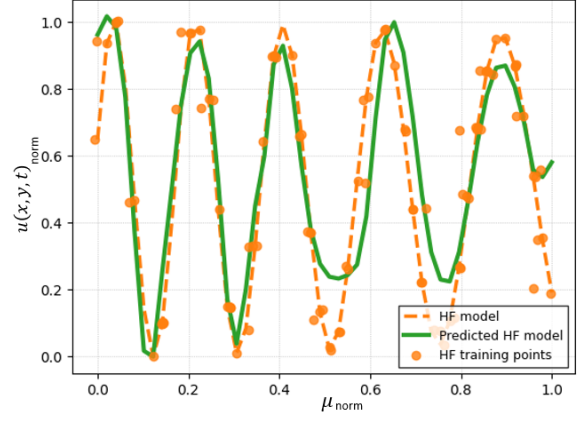
In order to confirm the quality of the method, a comparison with the corresponding low-fidelity and high-fidelity standalone networks must be performed. In fact, it is expected that for the same complexity, the multi-fidelity network is able to perform better than its single counterparts as it is able to extract information from both datasets. It can be seen that in this case, one can actually see an improvement by the 2-step network over the single network (Table 3).

	LF NN tested on LF set	LF NN tested on HF set	HF NN tested on HF set
MSE	0.02720	0.20011	0.09802
R^2	0.77399	-0.76781	0.13401

Table 3: Comparison of MSE and R^2 for single NN with LF and HF data. Same parameters as the first NN of the "2-step" case

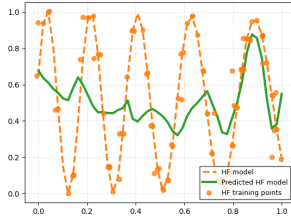


(a) LF network

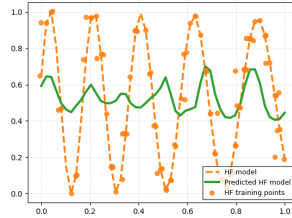


(b) HF network

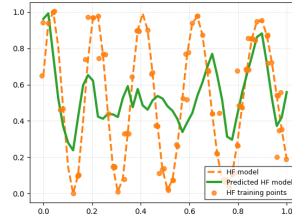
Figure 11: Representation of the prediction of the first (LF) Neural Network and of the second (HF) Neural Network in the best 2-step case ($n_{LF} = 46$ and $d = 1.00$) with $(x, y) = (2.23, 2.23)$ and $t = T = 80$.



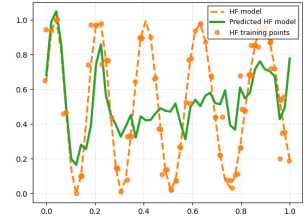
(a) $n_{LF} = 10, d = 0.00$



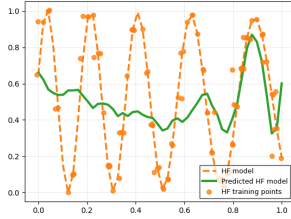
(b) $n_{LF} = 10, d = 0.50$



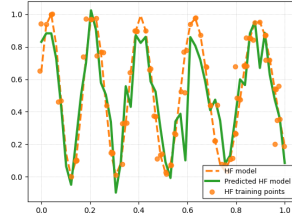
(c) $n_{LF} = 10, d = 0.75$



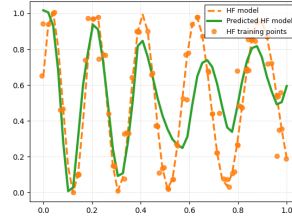
(d) $n_{LF} = 10, d = 1.00$



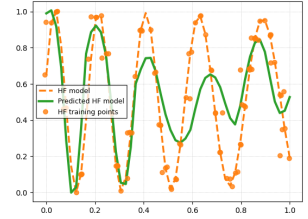
(e) $n_{LF} = 28, d = 0.00$



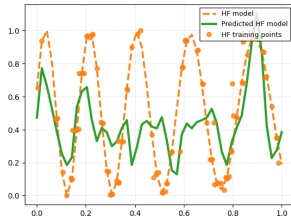
(f) $n_{LF} = 28, d = 0.50$



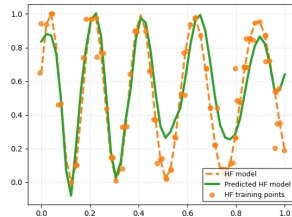
(g) $n_{LF} = 28, d = 0.75$



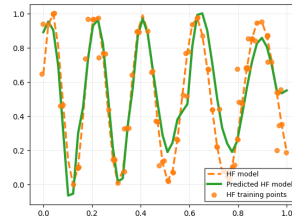
(h) $n_{LF} = 28, d = 1.00$



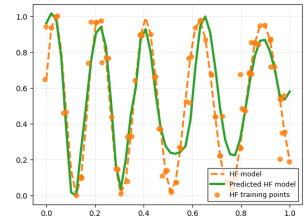
(i) $n_{LF} = 46, d = 0.00$



(j) $n_{LF} = 46, d = 0.50$

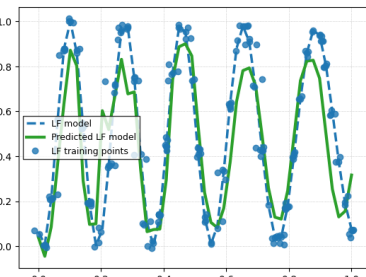


(k) $n_{LF} = 46, d = 0.75$

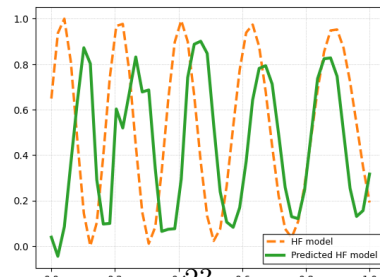


(l) $n_{LF} = 46, d = 1.00$

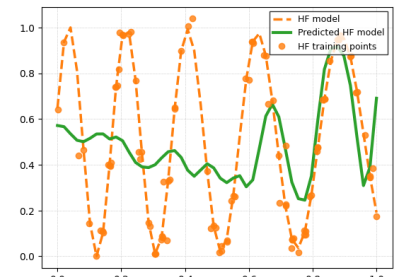
Figure 12: Collection of plots representing the surrogate 2-step model between normalized u (y -axis) versus a varying normalized value of reaction parameter μ (x -axis) for a varying discretization and diffusion parameter d .



(a) NN trained on low-fidelity dataset, comparison with LF set



(b) NN trained on low-fidelity dataset, comparison with HF set



(c) NN trained on high-fidelity dataset

Discretization	Diffusion	Test MSE	R ²
10	0.00	0.02335	0.81264
10	0.50	0.017741	0.85739
10	0.75	0.036607	0.71304
10	1.00	0.028136	0.76840
28	0.00	0.04611	0.62994
28	0.50	0.024452	0.78410
28	0.75	0.00522	0.95560
28	1.00	0.00371	0.96859
46	0.00	0.01112	0.91072
46	0.50	0.006162	0.94582
46	0.75	0.00658	0.94406
46	1.00	0.01223	0.89429

Table 1: Performance of the low-fidelity Network with respect to the low-fidelity data

Discretization	Diffusion	Test MSE	R ²
10	0.00	0.09502	0.16055
10	0.50	0.10235	0.09582
10	0.75	0.08552	0.24449
10	1.00	0.07716	0.31834
28	0.00	0.09512	0.15964
28	0.50	0.02434	0.78489
28	0.75	0.04437	0.60802
28	1.00	0.04878	0.56899
46	0.00	0.09043	0.20109
46	0.50	0.02160	0.80914
46	0.75	0.020267	0.82095
46	1.00	0.02213	0.80448

Table 2: Final Models Performance, "2-step" versus HF data

4.5. 3-step case

To take advantage of what was introduced in section 2.4.2, the multi-fidelity network can be enriched by introducing components that are able to capture behaviours of the data sets under consideration. In particular, in this case the introduction of a network NN_{per} (where "per" stands for periodicity, since tries to capture this behaviour), able to exploit the almost periodic shape of the output, has been proven to be useful. The introduced network is made of 2 hidden layers based on a custom activation function "sinusoidal activation"

$$f(x) = \sin^2(x). \quad (14)$$

This is able to capture the periodic behaviour of the relation. The quality of this introduction can be seen looking at the results in Table 4. Not only the R^2 and the MSE of the methods related to a finer discretization increased, but also using a simpler dataset based on a coarser discretization with $n_{LF} = 28$ is sufficient to obtain, in most cases, a meaningful relation. This can simplify by a lot the complexity of our regression, even if it comes from a more complex architecture. The results for all the combinations are reported in Figure 14.

4.6. Example II: Benchmark cases

4.6.1 Approximation of the chosen relations through multi-fidelity neural networks

To show the potential of both multi-fidelity construction and MCMC methods, we define very simple cases of parameter-dependent models. In these case, the LF and HF functions are correlated with different approximately linear and non-linear correlations. To show a diverse variety of cases, we will consider a basic case, a continuous function whose parameter δ defines the position in space of the function (translation parameter):

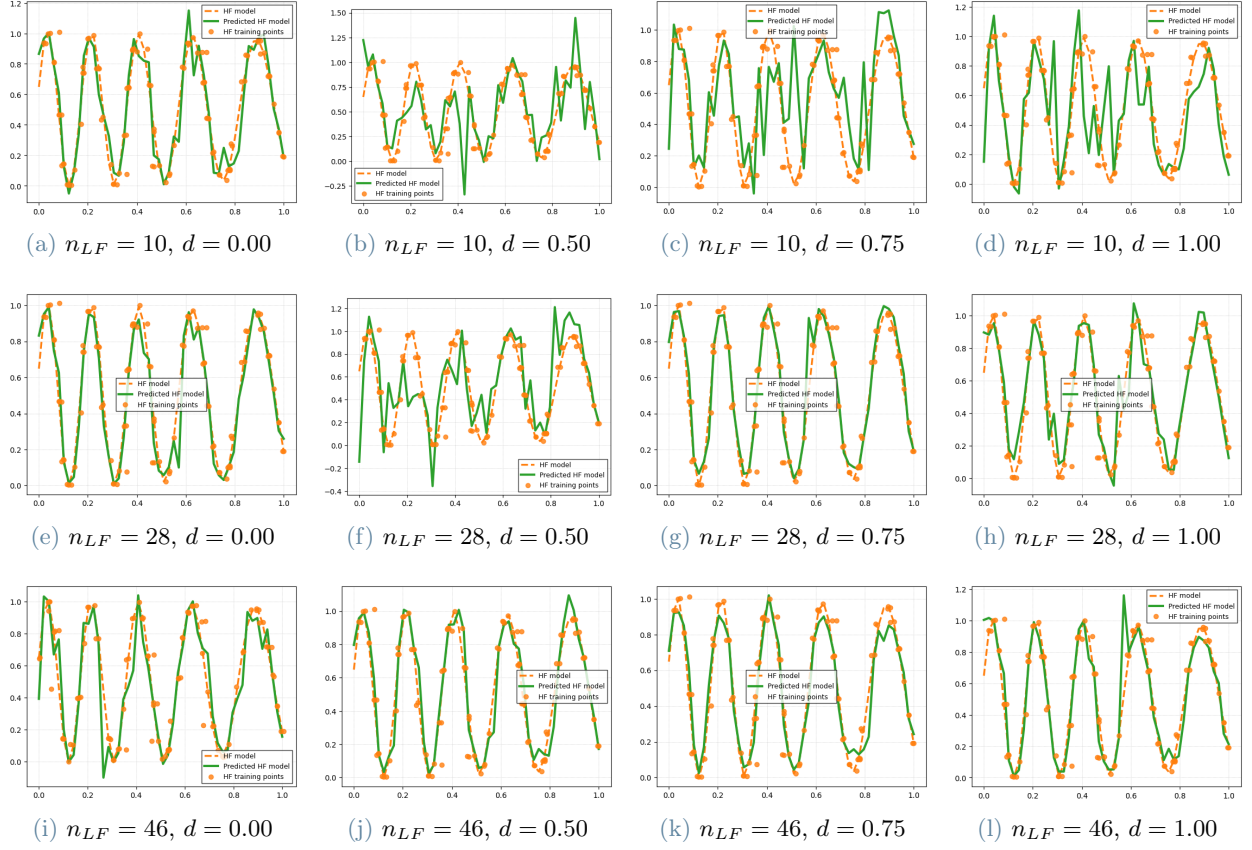


Figure 14: Collection of plots representing the surrogate 3-step model between normalized u (y -axis) versus a varying normalized value of reaction parameter μ (x -axis) for a varying discretization and diffusion parameter d .

Discretization	Diffusion	Test MSE	R^2
10	0.00	0.04658	0.58848
10	0.50	0.02284	0.79824
10	0.75	0.00208	0.98162
10	1.00	0.03683	0.67464
28	0.00	0.03423	0.69764
28	0.50	0.04844	0.57205
28	0.75	0.00754	0.93338
28	1.00	0.00356	0.96851
46	0.00	0.01859	0.83576
46	0.50	0.01253	0.88929
46	0.75	0.00580	0.94875
46	1.00	0.00752	0.93353

Table 4: Final Models Performance, "3-step" versus HF data. It is possible to notice a relevant improvement with respect to the results in Table 2.

$$f_{HF}(x, \delta) = \left(\frac{6 \cdot |x + \frac{\delta}{6}|}{5} - 2 \right)^2 \cdot \sin \left(\frac{12 \cdot |x + \frac{\delta}{6}|}{5} - 4 \right) \quad (15)$$

$$f_{LF}(x, \delta) = 0.5 \cdot \left(\frac{6 \cdot |x + 0.2 + \frac{\delta}{6}|}{5} - 2 \right)^2 \cdot \sin \left(\frac{12 \cdot |x + 0.2 + \frac{\delta}{6}|}{5} - 4 \right) + 10 \cdot \left(\frac{|x + 0.2 + \frac{\delta}{6}|}{5} - 0.5 \right) + 5 \quad (16)$$

a discontinuous case, in which a parameter defines the amplitude of the discontinuity

$$f_{LF}(x, \delta) = \left(0.5 \cdot \left(\frac{6x}{5} - 2 \right)^2 \cdot \sin \left(\frac{12x}{5} - 4 \right) + 10 \cdot \left(\frac{x}{5} - 0.5 \right) - 5 \right) \cdot \mathbb{I}(x < 2.5) \quad (17)$$

$$+ \left(3 + 0.5 \cdot \left(\frac{6x}{5} - 2 \right)^2 \cdot \sin \left(\frac{12x}{5} - 4 \right) + 10 \cdot \left(\frac{x}{5} - 0.5 \right) - 5 + \delta \right) \cdot \mathbb{I}(x > 2.5),$$

$$f_{HF}(x, \delta) = \left(2 \cdot f_{LF}(x, \delta) - 20 \cdot \frac{x}{5} + 20 \right) \cdot \mathbb{I}(x < 2.5) \quad (18)$$

$$+ \left(4 + 2 \cdot f_{LF}(x, \delta) - 20 \cdot \frac{x}{5} + 20 + \delta \right) \cdot \mathbb{I}(x > 2.5),$$

and a wave case, in which a parameter denotes the frequency of the oscillations:

$$f_{LF}(x, \delta) = \sin(\delta \cdot x) \quad (19)$$

$$f_{HF}(x, \delta) = \left(\frac{x}{5} - \sqrt{2} \right) \cdot (f_{LF}(x, \delta))^2. \quad (20)$$

They are all illustrated in Figure 15. In this cases, it is possible to create a simple but effective "2-step" NN that gives us the opportunity to see again that, if a correlation between LF and HF dataset is found and exploited by the NN, then a gain is achieved. The final results for this case are collected in Figure 16.

The following architectures NNs were configured with distinct hyperparameters tailored to their respective tasks. The NN_{LF} utilized a learning rate of 0.0255, which balances the rate of convergence and stability during training. The kernel weights were initialized using the `glorot_uniform` method, an approach designed to maintain the variance of outputs across layers, thereby preventing gradient vanishing or explosion. The `Adam` optimizer was employed for its adaptive learning rate capabilities, which enhance performance across a wide range of problems.

For NN_{HF} (the second component of the 2 step network), a different configuration was chosen to address its specific requirements. The kernel weights were initialized with a uniform distribution, ensuring that each neuron starts with weights drawn from the same range. An L2 regularization weight of 0.0001 was applied to prevent overfitting by penalizing large weights. The learning rate was set to 0.0709, a higher value compared to the first network, to expedite convergence for this particular model. This network was designed with 74 nodes, providing a balance between model complexity and computational efficiency. The `Adamax` optimizer, a variant of Adam that is more robust to outliers in the gradient, was selected to further stabilize training.

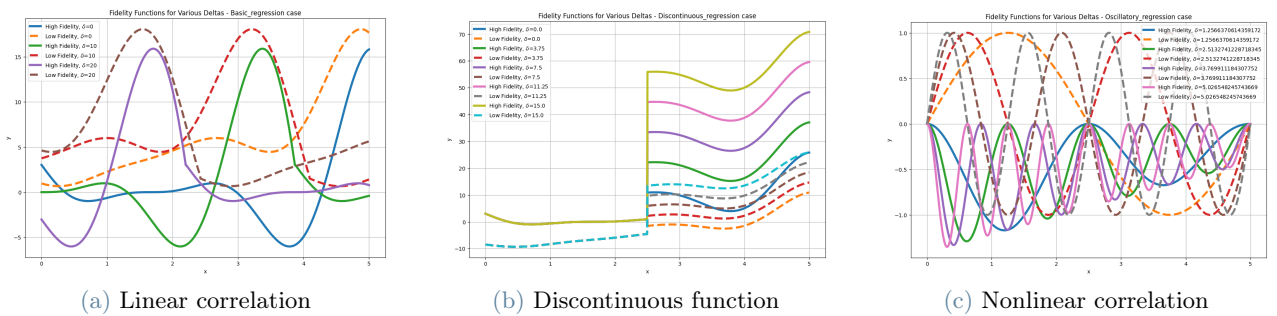


Figure 15: Benchmark functions considered

As these are still rather simple cases, the networks used are "two-step" networks, i.e. in which one has a first network trained on LF data, followed by a second, poorer network, which, starting from the HF data and the output of the first network evaluated on the same data, allows one to approximate the desired output. The used parameters are the same in all cases. The performance are summarized in table 5. Given the high quality of the final result, the idea is to use the relation we have built with the "2-step" NN as a forward model for the inverse problem.

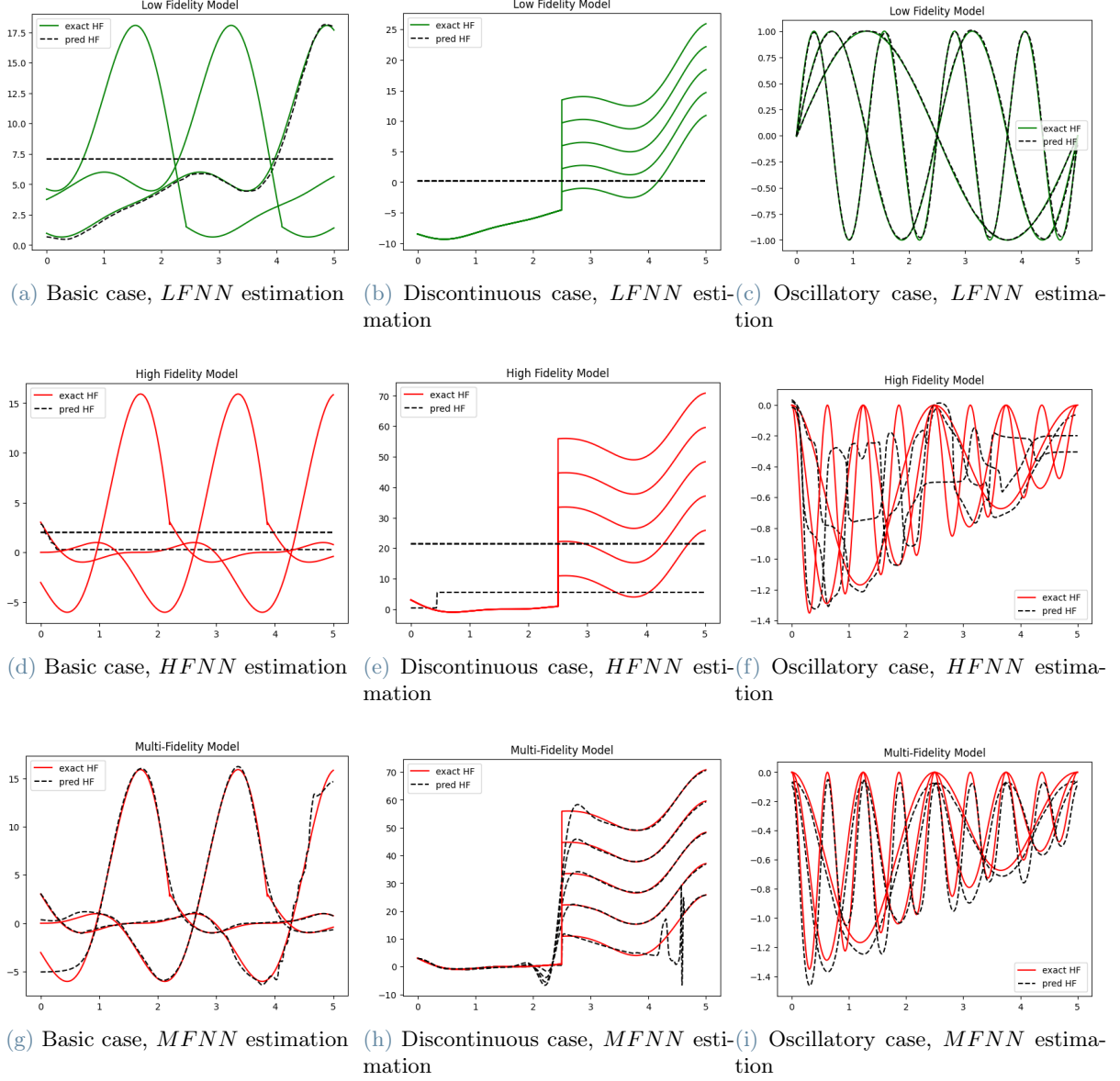


Figure 16: Starting from the top, neural network trained and evaluated on LF data, neural network trained and evaluated on HF data and "2-step" neural network's estimate on high-fidelity data.

	NN_{LF}	NN_{HF}	NN_{MF}
MSE	54.5355	29.0067	0.1704
R^2	-0.8550	0.0133	0.9942
MSE	696.2389	394.2605	9.0680
R^2	-0.6444	0.0687	0.9785
MSE	0.9567	0.0556	0.0102
R^2	-6.1606	0.5831	0.9232

Table 5: Comparison of MSE and R^2 for different model configurations across Basic, Discontinuous, and Oscillatory cases.

4.6.2 Parameter estimation

Thanks to the system described in Section 4.2, it is possible to exploit the forward models introduced in Section 4.6 to deduce, given some observations, which parameter most likely correspond to the provided result.

This is particularly useful when it is very difficult or impossible to reverse a relationship, but it is easier to have a forward evaluation. In our case, however, a further approximation is inserted, as we are not using the theoretical relation as a forward model, but instead the estimate defined by the previously introduced multi-fidelity network. Intuitively, the more precise the model used, the better the approach works. The unavoidable inaccuracies of the model at our disposal can therefore be interpreted as artificial additional noise hampering the observations. In this simplified case scenario, we investigate only the parameters on which the architecture has been trained and tested. Since, almost everywhere, the networks’ estimate are the same with respect to the real ones, we expect good performance almost in every case. To do so, we follow an **hpo** procedure as described in section 4.2, choosing the basics Metropolis-Hastings algorithm and testing a wide range of number of observations, noise’s covariances (σ_{noise}^2), likelihood’s standard deviations ($\sigma_{likelihood}$), covariances and scalings of the proposal (respectively, $\sigma_{proposal}^2$ and s). Through this algorithm, it was possible to find a combination of values for which a very precise estimate of the parameter considered in this first simple case is achieved.

To give an example, consider the discontinuous context in which the jump parameter δ has to be identified. Through the hyper-parameter search algorithm including the aforementioned elements to be investigated, it is possible to find a good combination of values for the search. Since this is a simple case, a short search is sufficient, the results are written in Table 6. As previously mentioned, HPO has as reference the relative error with respect to the real parameter δ and the expected value of the estimated posterior distribution. As a consequence, the goal of the algorithm is to reduce this difference as much as possible. However, an MCMC approach cannot be said to be healthy only insofar as it approaches the desired value. It is indeed possible, for example, that the generated chains are not characterised by appropriate mixing and thus do not adequately explore the parameter space, and that the estimated value is merely dictated by the fact that the prior has randomly identified a value similar to the desired one. For this reason, it is essential to assess diagnostic estimates issued by the algorithm. Just to give an example, the results associated with the above-mentioned case are given in Figure 17 and it is possible to notice that the autocorrelation suddenly drops at zero for both chains and that the chains show a good behaviour when exploring the parameter space. Moreover, $\hat{R} = 1.01$ is a good indication of convergence.

iter	target	N_{obs}	s	$\sigma_{likelihood}$	σ_{noise}^2
1	-0.04373	37	1.451	1.232	0.3997
2	-0.03893	33	0.656	0.5581	0.4665
3	-0.02187	42	1.208	0.5206	0.4925
4	-0.03173	42	1.232	0.5482	0.447
5	-0.03147	48	1.325	0.8949	0.3646

Table 6: Example of short HPO output. The first 3 results are the initial points, while the remaining 2 are attempted improvements of the best of the initial points. The target column denotes the opposite of the relative error to maximize. In this case, we are considering the discontinuous example, with $\delta = 3.75$ as real parameter.

4.7. Generalization

In order to further push the capabilities of the multilevel networks studied, and to investigate how reliable the algorithms implemented for the inverse problem are, we present a more general case where, following an optimisation process of the hyper-parameters (see section 4.1.4), a larger and more varied test set is obtained compared to the training set and the test set of the previous case. As a result of the process of finding the optimal hyperparameters, the identified combination improves on what was done previously. A couple of examples are presented, for the Basic and Discontinuous cases respectively.

As mentioned before, an HPO has been necessary in this case in order to achieve a result with appropriate MSE and R^2 values. To do so, it has been decided to keep the first network as in the previous cases and to optimize the hyperparameters only of the second network. Even if the complexity of this case is higher than before, the Basic case, where the parameter δ denotes a translation has $R^2 = 0.924$ and $MSE = 2.30$, while the discontinuous $R^2 = 0.970$ and $MSE = 12.454$. In this case, given the good performance but not the scarce correspondence of the model identified with the actual mathematical relationship, it is more interesting to study the validity of our Metropolis-Hastings implementation in order to verify its robustness and its ability to guess the parameter sought even in non-ideal situations. A couple of results are shown in Figure 20 as an example. To test this case, the parameters of the test set has been considered, in order not to necessarily use elements of the training set, but also solutions related to parameters not considered in the learning phase.

Together with those results, obtained through **TinyDA**, we show a couple more achieved exploiting **CUQIpy**. As

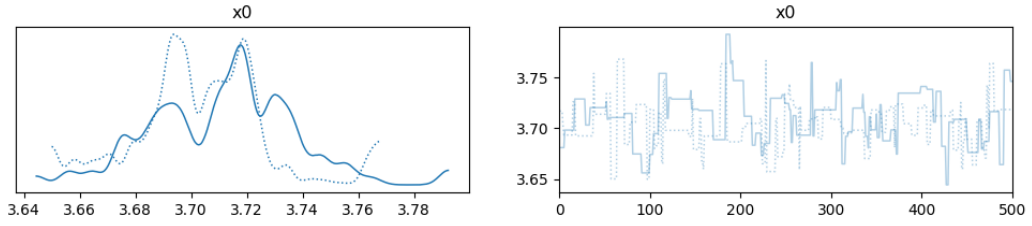


Figure 17: Trace plots related to the discontinuous case. Real parameter is $\delta = 3.75$. Two chains of length 1500 have been considered, burn-in $B = 1000$. The elements for an optimal performance are the ones in Table 6 related to the smallest relative error.

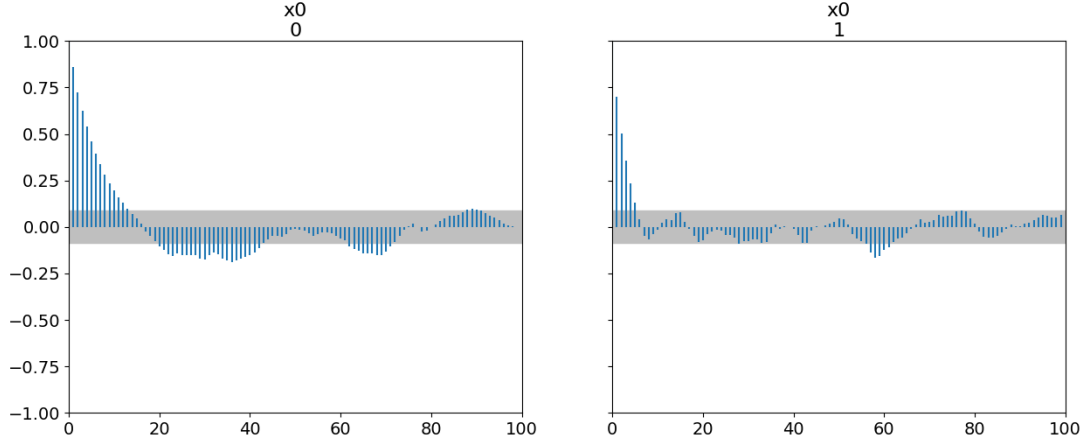


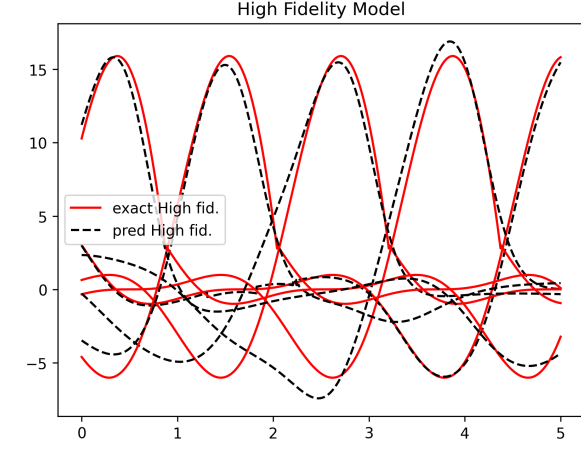
Figure 18: Autocorrelation of the 2 Markov chains.

explained in section 4.2, the definition of a method using one library or the other is basically the same. Looking at Figure 22, it is possible to notice that also with this library we are able to achieve very good performance, equivalent to the ones with TinyDA. The presence of this second library allows us to have other methods and to show possible expansions of the current implementation. For example, thanks to CUQIpy, it was also possible to test the application of preconditioned Crank-Nicholson (pCN) which, using the same parameters as the previously mentioned example, estimates the parameter to be $\hat{\delta} = 21.54$, very similar to the real one.

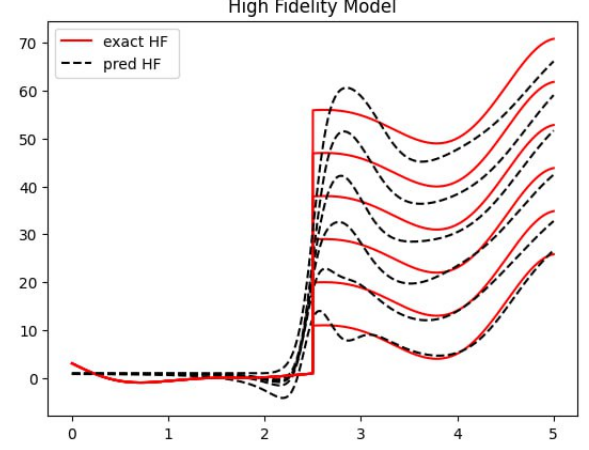
5. Example III: Ordinary Differential Equations, Lotka-Volterra case

In the following, we consider a procedure similar to the one described in sections 4.6 and 4.7, but in a different and more complex context. Here we study systems of ordinary differential equations solved through numerical methods. The difference between coarse data and fine data is therefore related to the discretization and the type of solver being considered. For these tests, two cases of increasing complexity are considered, namely two Lotka-Volterra systems. In the first case, we will exploit a system of 2 equations linked by one parameter, while in the second case, a system of 3 equations with 4 parameters. These examples are therefore more complex than the previous cases, both in terms of the number of parameters and relations taken into consideration and because of the non-linear nature of the solutions of the system.

The Lotka-Volterra system, also known as the predator-prey model, is a set of first-order, non-linear differential equations used to describe the dynamics between two interacting species: a predator and its prey. The model captures the oscillatory nature of predator and prey populations over time. It assumes that the prey population grows exponentially in the absence of predators, while the predator population declines in the absence of prey. The interaction terms represent the effect of predation on prey and the corresponding growth in the predator population, leading to coupled equations that exhibit periodic solutions under certain conditions. Various versions of the Lotka-Volterra system exist, depending on the number of parameters and the types of interactions considered, allowing the model to be adapted for different ecological scenarios and complexities. This system serves as a foundational model in theoretical ecology for understanding population dynamics and species interactions [58].



(a) Basic case performance on test set, with parameters $\delta \in (0, 28)$



(b) Discontinuous case performance on test set, with parameters $\delta \in (0, 15)$

Figure 19: Approximation of benchmark functions with varying parameters.

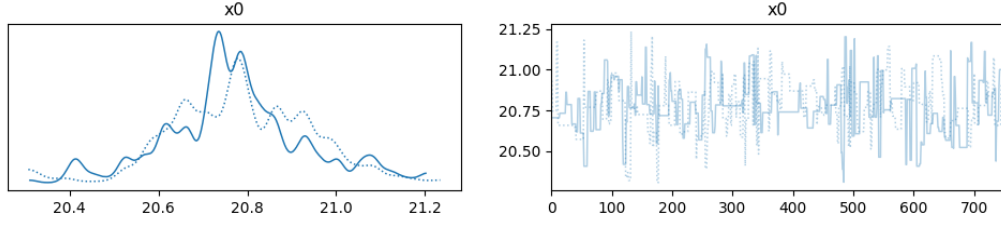


Figure 20: Trace plots related to the basic case. Real parameter is $\delta = 21$. Two chains of length 2000 have been considered, burn-in $B = 1250$. The elements for an optimal performance have been found to be $N_{obs} = 37$, $s = 2.911$, $\sigma_{likelihood} = 2.5$ and $\sigma_{noise}^2 = 0.15$.

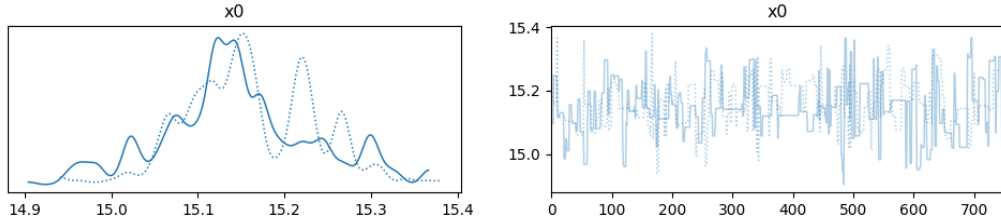


Figure 21: Trace plots related to the discontinuous case. Real parameter is $\delta = 15$. Two chains of length 2000 have been considered, burn-in $B = 1250$. The elements for an optimal performance have been found to be $N_{obs} = 37$, $s = 1.5$, $\sigma_{likelihood} = 1.331$ and $\sigma_{noise}^2 = 0.0511$.

5.1. Classic random-walk Metropolis-Hastings alternatives

As mentioned before in Section 3.2, Metropolis-Hastings algorithm is a reference for Markov Chain Monte Carlo (MCMC) methods, designed to sample from complex probability distributions. However, despite its foundational role, MH has several limitations, particularly when applied to high-dimensional problems. One of the primary issues with MH is its reliance on random walk behavior for generating proposals. In high-dimensional spaces, this approach can lead to extremely slow exploration of the target distribution. The algorithm tends to take small steps, often resulting in poor mixing, where the chain gets "stuck" in certain regions of the space for long periods. This inefficiency worsens as the dimensionality increases, making it harder for the chain to move through the space effectively. Moreover, the acceptance rate of proposed moves in the MH algorithm typically decreases with increasing dimensionality, further hindering its performance. The efficiency of MH is also highly dependent on the choice of the proposal distribution. In high-dimensional spaces, finding an optimal proposal distribution becomes a complex and often labor-intensive task, requiring careful tuning of parameters like step size. Poor tuning can result in either too many rejected proposals or a chain that fails to explore the space

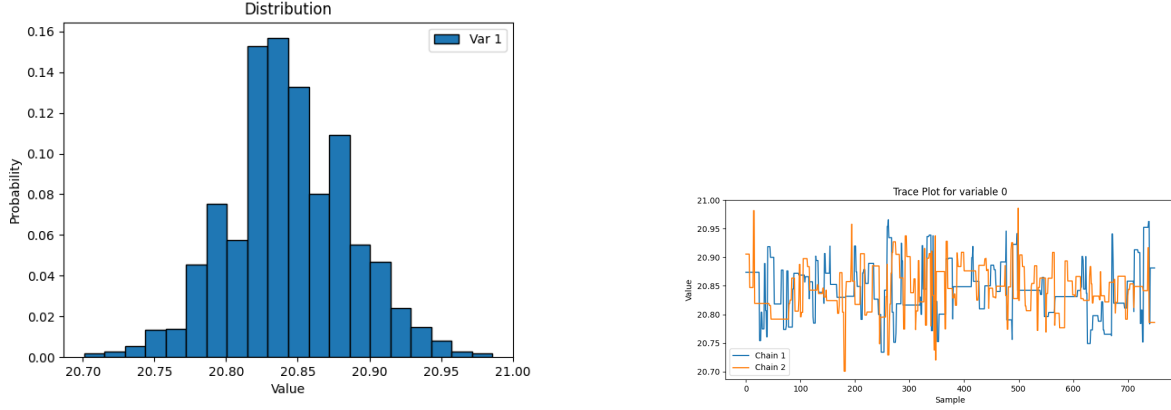


Figure 22: Trace plot related to the basic case. Real parameter is $\delta = 21$. Two chains of length 2000 have been considered, burn-in $B = 1250$. The parameters for an optimal performance have been found to be $N_{obs} = 31$, $\sigma_{noise}^2 = 0.09649$, $\sigma_{proposal} = 0.6484$ and $s = 1$.

thoroughly, both of which undermine the algorithm’s effectiveness.

Given these limitations, several alternatives to random-walk Metropolis-Hastings have been developed to improve sampling efficiency, especially in high-dimensional settings. Methods like Hamiltonian Monte Carlo (HMC) [8] utilize gradient information to propose more efficient moves, reducing random walk behavior and improving convergence. A popular variant of HMC, the No-U-Turn Sampler (NUTS) [29], automatically determines an optimal path length for exploration, removing the need for manual tuning and enhancing performance. Another notable alternative is the Preconditioned Crank-Nicholson (pCN) algorithm [49], which is particularly effective in high-dimensional spaces. The pCN method modifies the standard Metropolis-Hastings approach by introducing a preconditioner that adjusts the proposal distribution, allowing for more effective exploration of the target distribution. This results in more stable and efficient sampling, especially when dealing with complex or high-dimensional problems. These advanced techniques address some of the shortcomings of the classical random-walk Metropolis-Hastings algorithm, making them more suitable for complex problems often encountered in modern Bayesian inference. Some of the mentioned methods are implemented in the presented code, however we want to focus on some newer Adaptive Metropolis [28] and Delayed acceptance MCMC methods [14], shortly described hereafter.

5.1.1 Adaptive Metropolis Hastings

The Adaptive Metropolis (AM) algorithm, presented in [28], improves efficiency by dynamically adjusting the proposal distribution during the sampling process. More precisely (see algorithm 2), it updates the covariance matrix of the Gaussian proposal distribution based on the history of all previously accepted states. This adaptation allows the proposal distribution to better approximate the shape and scale of the target distribution over time. By continuously refining the proposal distribution, the AM algorithm enhances the exploration of the target distribution, leading to faster convergence and more efficient sampling, especially in high-dimensional spaces.

Algorithm 2 Adaptive Metropolis (AM) Algorithm

- 1: **Input:** Target distribution $\pi(x)$, Initial state x_0 , Initial covariance C_0 , Scaling parameter $s_d = \frac{2.4^2}{d}$, Small constant $\epsilon > 0$, Number of iterations N
- 2: **Output:** Sequence of states $\{x_t\}_{t=0}^N$ approximating $\pi(x)$
- 3: Initialize $x_0 = x_{\text{initial}}$
- 4: **for** $t = 1$ to N **do**
- 5: Sample $y_t \sim \mathcal{N}(x_{t-1}, C_{t-1})$
- 6: Compute the acceptance probability:

$$\alpha = \min \left(1, \frac{\pi(y_t)}{\pi(x_{t-1})} \right)$$

- 7: Sample $u \sim \text{Uniform}(0, 1)$
- 8: **if** $u \leq \alpha$ **then**
- 9: Set $x_t = y_t$
- 10: **else**
- 11: Set $x_t = x_{t-1}$
- 12: **end if**
- 13: Update the covariance matrix:

$$C_t = s_d \times \text{Cov}(x_0, x_1, \dots, x_t) + \epsilon I_d$$

- 14: **end for**
 - 15: **Return:** Sequence $\{x_t\}_{t=0}^N$
-

5.1.2 Delayed acceptance Monte Carlo Markov Chain

Delayed Acceptance Markov Chain Monte Carlo is an advanced MCMC technique designed to enhance computational efficiency, particularly when evaluating the target distribution is computationally expensive. The algorithm operates in a two-stage acceptance procedure. In the first stage, a proposal is evaluated against a surrogate target distribution, which is a computationally cheaper approximation of the true target. If the proposal passes this initial acceptance, it then proceeds to the second stage, where it is evaluated against the actual target distribution using the Random-Walk Metropolis-Hastings criterion. This method reduces computational cost by allowing early rejection of proposals that are unlikely to be accepted by the true target distribution, thus avoiding unnecessary evaluations of expensive computations.

Algorithm 3 Delayed Acceptance (DA)

- 1: **function:** $[\theta^1, \dots, \theta^N] = \text{DA}(\pi_F(\cdot), \pi_C(\cdot), q(\cdot|\cdot), \theta^0, N)$
- 2: **input:** target (fine) density $\pi_F(\cdot)$, approximate (coarse) density $\pi_C(\cdot)$, proposal kernel $q(\cdot|\cdot)$, initial state θ^0 , number of steps N
- 3: **output:** ordered list of states $[\theta^1, \dots, \theta^N]$ (or just the final state θ^N)
- 4: **for** $j = 0$ to $N - 1$ **do**
- 5: Given θ^j , generate proposal ψ by invoking one step of MH Alg.1 for coarse target π_C :

$$\psi = \text{MH}(\pi_C(\cdot), q(\cdot|\cdot), \theta^j, 1)$$

- 6: Accept proposal ψ as the next state, i.e., set $\theta^{j+1} = \psi$, with probability:

$$\alpha(\psi|\theta^j) = \min \left\{ 1, \frac{\pi_F(\psi)q_C(\theta^j|\psi)}{\pi_F(\theta^j)q_C(\psi|\theta^j)} \right\}$$

- 7: otherwise reject proposal ψ and set $\theta^{j+1} = \theta^j$
 - 8: **end for**
-

The Delayed Acceptance Markov Chain Monte Carlo (DA-MCMC) algorithm employs a hierarchical approach to MCMC by introducing different fine and coarse states for proposal evaluation. In this method, the fine state represents the detailed, high-dimensional target distribution, while the coarse state is a simplified, lower-dimensional approximation. The algorithm first assesses a proposal using the coarse state, which is computationally less expensive. If the proposal is accepted at this level, it then undergoes a more thorough evaluation in the fine state against the actual target distribution. This two-step process optimizes computational efficiency by focusing resources on the most promising proposals. The algorithm maintains detailed balance by ensuring that transitions between states, guided by the surrogate distribution at the coarse level, are aligned with the overall target distribution at the fine level. Given this 2-levels formulation, the idea is to generalize this approach in order to leverage the potentiality of different precision levels. This is introduced in Multilevel Delayed Acceptance MCMC.

5.1.3 Multilevel delayed acceptance MCMC (MLDA)

Multilevel methods, particularly Multilevel MCMC techniques, have become increasingly prominent in computational science for their ability to enhance efficiency in complex Bayesian inference problems. These methods leverage a hierarchy of models or discretizations, refining the accuracy of approximations at each level while reducing computational costs. Over time, various evolutions of Multilevel MCMC have been developed, each introducing different strategies for balancing the trade-off between computational efficiency and statistical accuracy. The evolution of these methods has been driven by advancements in understanding how to optimally combine information across multiple levels, leading to significant improvements in both convergence rates and scalability [21][22][10][36].

Algorithm 4 Multilevel Delayed Acceptance (MLDA)

- 1: **Input:** Target densities $\pi_0(\cdot), \dots, \pi_\ell(\cdot)$, proposal densities $q_0(\cdot)$ and $q_{1,F}(\cdot), \dots, q_{\ell,F}(\cdot)$, probability mass functions $p_1(\cdot), \dots, p_\ell(\cdot)$, initial state θ_ℓ^0 , current level index ℓ , number of steps N
- 2: **Output:** Sequence of states $\{\theta_\ell^j\}_{j=0}^N$ at level ℓ (or just the final state θ_ℓ^N)
- 3: **for** $j = 0$ to $N - 1$ **do**
- 4: Draw the subchain length $n_\ell \sim p_\ell(\cdot)$ for level $\ell - 1$
- 5: Starting at $\theta_{\ell,C}^j$, generate a subchain of length n_ℓ on level $\ell - 1$:
- 6: **if** $\ell = 1$ **then**
- 7: Use the random-walk Metropolis–Hastings algorithm to generate the subchain:

$$\psi_C = \text{MH} \left(\pi_0(\cdot), q_0(\cdot, \cdot), \theta_{\ell,C}^j, n_1 \right)$$

- 8: **else**
- 9: Generate the subchain by (recursively) calling MLDA:

$$\psi_C = \text{MLDA} \left(\{\pi_k^\ell\}_{k=0}^{\ell-1}, q_0(\cdot), \{q_k, F\}_{k=1}^{\ell-1}, \{p_k\}_{k=1}^{\ell-1}, \theta_{\ell,C}^j, \ell - 1, n_\ell \right)$$

- 10: **end if**
- 11: Draw the fine-mode proposal $\psi_F \sim q_{\ell,F}(\cdot \mid \theta_{\ell,C}^j)$
- 12: Accept proposal $\psi = (\psi_F, \psi_C)$ as the next sample, i.e., set $\theta_\ell^{j+1} = \psi$, with probability:

$$\alpha(\psi \mid \theta_\ell^j) = \min \left(1, \frac{\pi_\ell(\psi) \pi_{\ell-1}(\psi_C)}{\pi_\ell(\theta_\ell^j) \pi_{\ell-1}(\psi_C)} \right)$$

- 13: Otherwise, reject and set $\theta_\ell^{j+1} = \theta_\ell^j$
 - 14: **end for**
 - 15: **Return:** Sequence $\{\theta_\ell^j\}_{j=0}^N$
-

When coarser levels are evaluated, they might compromise the use of the algorithm, for instance making impossible to accept the proposed values, drastically decreasing mixing and slowing convergence. For this reason, in conjunction with a multi-level approach, it is useful to use Adaptive Metropolis Hastings (see Section 5.1.1), or another adaptation method to have an adequate acceptance rate.

5.2. Lotka-Volterra MF-Neural Networks

As mentioned in the introduction to this section, here the following two Lotka-Volterra systems will be studied:

$$\begin{aligned}\frac{dy_1}{dt} &= y_1 (\mu - 0.1y_1 - 0.5y_2 - 0.5y_3), \\ \frac{dy_2}{dt} &= y_2 (-\mu + 0.5y_1 - 0.3y_3), \\ \frac{dy_3}{dt} &= y_3 (-\mu + 0.2y_1 + 0.5y_2).\end{aligned}\tag{21}$$

$$\begin{aligned}\frac{dy_1}{dt} &= ay_1 - by_1y_2, \\ \frac{dy_2}{dt} &= cy_1y_2 - dy_2.\end{aligned}\tag{22}$$

To create some proper Multi-Fidelity Neural Networks we follow a similar procedure to the one tested in section 4.6. The first step is the definition of both the low-fidelity and high-fidelity dataset.

5.2.1 Multi-fidelity datasets through different numerical schemes

To build both low-fidelity and high-fidelity dataset, different levels of precision are required in solving the underlying ordinary differential equations (ODE). The idea is to use various numerical methods and discretization approaches. For example, for the high-fidelity dataset, higher-order methods such as Runge-Kutta or multi-step methods, coupled with finer discretization, can be employed to achieve greater accuracy. Conversely, the low-fidelity dataset can be generated using simpler methods like Euler's method and coarser discretization, which provide sufficient accuracy at a reduced computational cost [45]. By selecting the appropriate numerical techniques and discretization levels (avoiding instabilities for the coarsest case), one can efficiently generate datasets with the desired precision, ensuring a balance between accuracy and computational efficiency. An example with varying discretizations is shown in section 4.4. For this specific case, the second order Runge-Kutta method for the high-fidelity dataset and the Forward Euler method for the low-fidelity dataset have been used.

5.2.2 Implementation of the ODE model

In order to be able to generate the data, a special `SystemSolver` class was created, which allows the solution of an ODE to be saved, visualised and, above all, quickly provided at the desired level of discretization and precision. ODEs are defined as small classes derived from `SystemSolver`. This choice, in addition to ordering the cases studied, has the aim of creating a system that is as generalizable as possible, in which it is possible to include both numerical methods and ODE systems while making almost no changes to the class itself.

5.2.3 One parameter case

The case denoted by a single parameter is particularly interesting as we are in the presence of several inputs and outputs.

This then makes it possible to create multi-fidelity networks in which the number of elements passed from one architecture to another becomes considerable. Given the structure of the system of ODEs (24), the network consists of two inputs (i.e., the time component and the parameter μ). Considering the 2-step network, there will therefore be a 3-dimensional output from the first network which will imply the fact that the hf network consists of as many as 5 inputs, and three outputs. This, is reflected in a more complicated structure to process. Good performance even in this context would validate the overall quality of the multi-fidelity approach. The dataset was created in the class defined in section 5.2.2. For the high fidelity component, a Runge-Kutta method with discretisation $h = 0.02$ on a domain $[0, T]$ with $T = 8$ was chosen. The parameter $\mu \in [1, 5]$. For the low-fidelity case, on the other hand, a forward Euler scheme with $h = 0.05$ is considered. Examples of datasets for different parameter choices are shown in Figure 24. It can be seen that, although there is a relationship between the low-fidelity component and the high-fidelity component, there are differences in both amplitude and oscillation dictated by the fact that the Euler method is not able to capture the complex, non-linear behaviour of the system with great accuracy. To model the phenomenon, a multi-fidelity structure is again used. Specifically, we opt for a 2-step case analogous to those mentioned before. In this case, hyperparameter optimisation was exercised over the multi-fidelity structure as a whole, and not only on the second network (hence, optimizing the networks in their proper order).

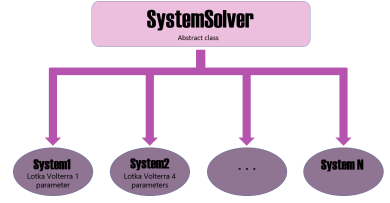
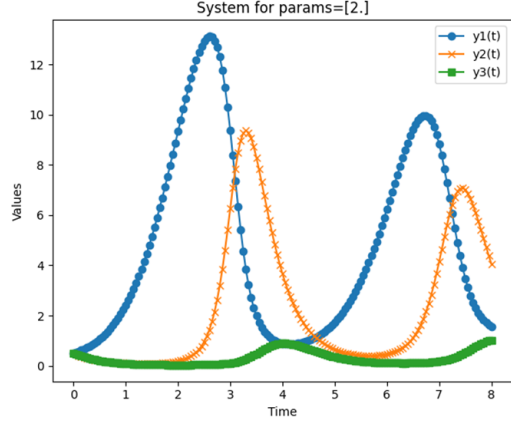
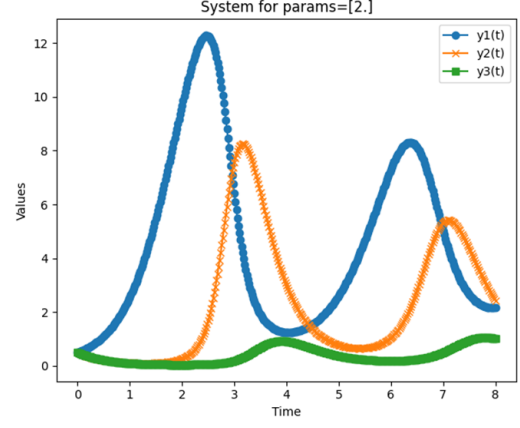


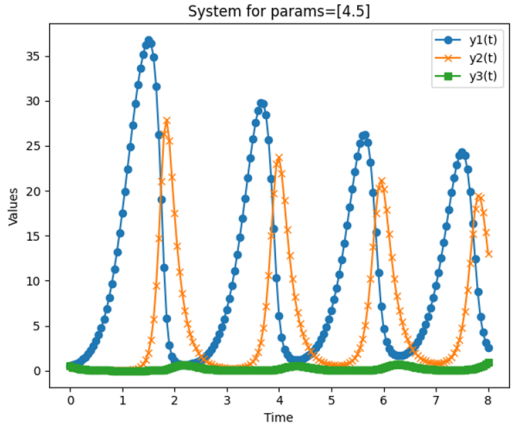
Figure 23: Graphical simplification of classes to handle numerical approximation methods and systems of ODEs.



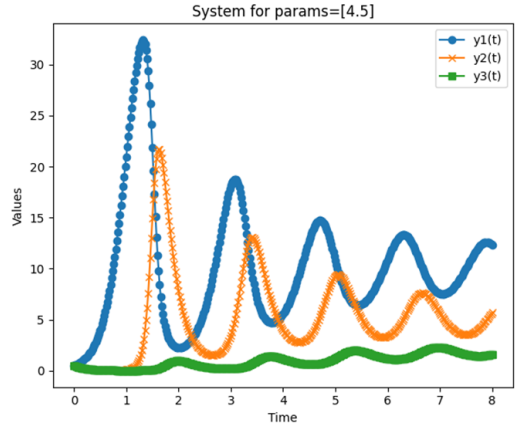
(a) Low-fidelity (Euler method) solution of system (22)



(b) High-fidelity (Runge-Kutta method) solution of system (22)



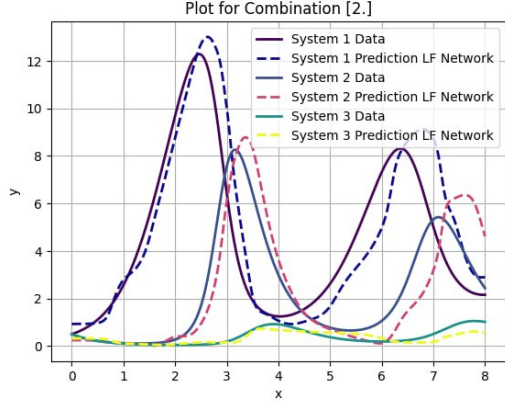
(c) Low-fidelity (Euler method) solution of system (22)



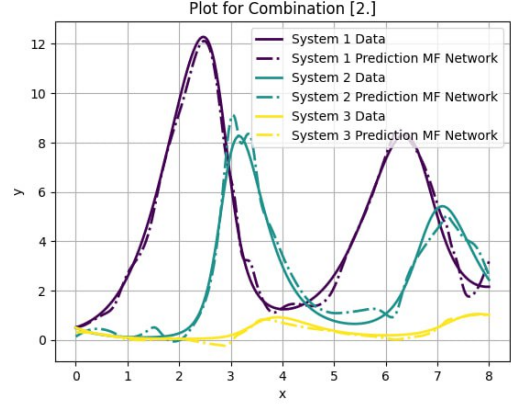
(d) High-fidelity (Runge-Kutta method) solution of system (22)

Figure 24: Euler and Runge-Kutta approximations of 1 parameter Lotka-Volterra system

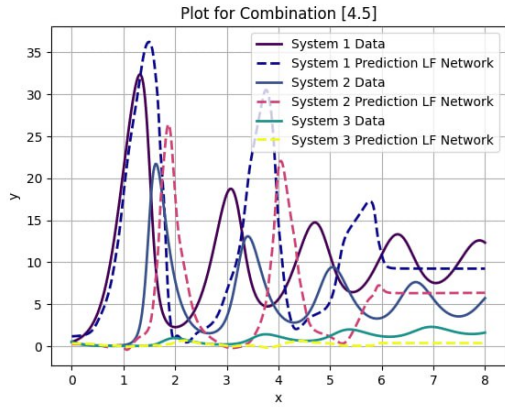
The training has been made with data related with 9 parameter values in the given range $[1, 5]$, the validation on a validation set with 6 parameters and the test with 15. As a result, an $MSE = 0.5005$ and $R^2 = 0.9761$ are reached. A couple of examples are shown in Figure 25.



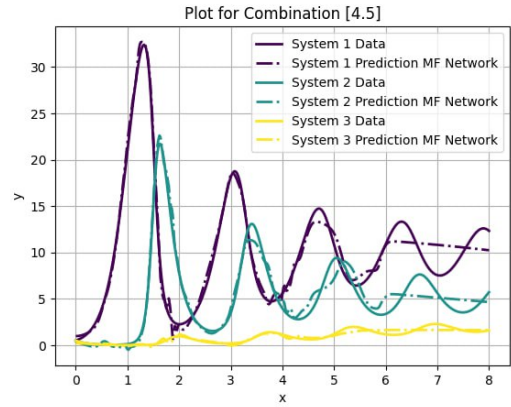
(a) Prediction of the first (LF) network against real solution



(b) Prediction of the MF network against real solution



(c) Prediction of the first (LF) network against real solution



(d) Prediction of the MF network against real solution

Figure 25: Examples of Lotka-Volterra approximations through components of "2-step" neural networks

5.2.4 Parameter μ estimation

Even for this case, although more complex than the previous ones due to the multitude of inputs and outputs and given the complexity of the relationship under consideration, it is possible to use the structure described in Section 4.2. In fact, the library implemented is constructed in such a way that, whatever the dimensionality of the problem or the quantity of parameters to be estimated, these are manageable without the user making any particular modifications (other than providing a quantity of averages and covariances for the priors that accords with the number of parameters involved). Tests were also carried out in this case, both with `CUQIPy` and `tinyDA`. In fact, starting from a simple case, it can be seen that the `CUQIPy` library is well suited to dealing with this problem. For example, one would like to try to estimate the behaviour per real parameter $\mu = 1$. To this end, following the optimisation process, it was possible to find a combination of parameters that allowed for a very small error associated with good behaviour of the Markov chains. The example is shown in Figure 26.

However, given the large number of proposal algorithms implemented, it would be ideal to make a comparison in order to be able to see which performs better. In particular, a thought for optimising the performance of our backward model arises by looking at the forward model. In fact, this allows us to have different models at our disposal, which tend to be more and more accurate and complex. This complexity is also reflected in the backward case, as the evaluation of the forward model becomes slightly more expensive as the network levels increase. This is due to the fact that, in order to go from input to output with the network, it is necessary to evaluate all the components that make up the network one after the other, which drives up the computational weight. Therefore, it would be ideal to possess a method that takes advantage of the many patterns that constitute multi-fidelity structures. For this purpose, the idea is to use the structure created to give the possibility of having a multilevel structure, with a number of levels equal to the number of surrogate models that constitute the network. Since in this case 2 levels are involved, the idea is to create a delayed acceptance

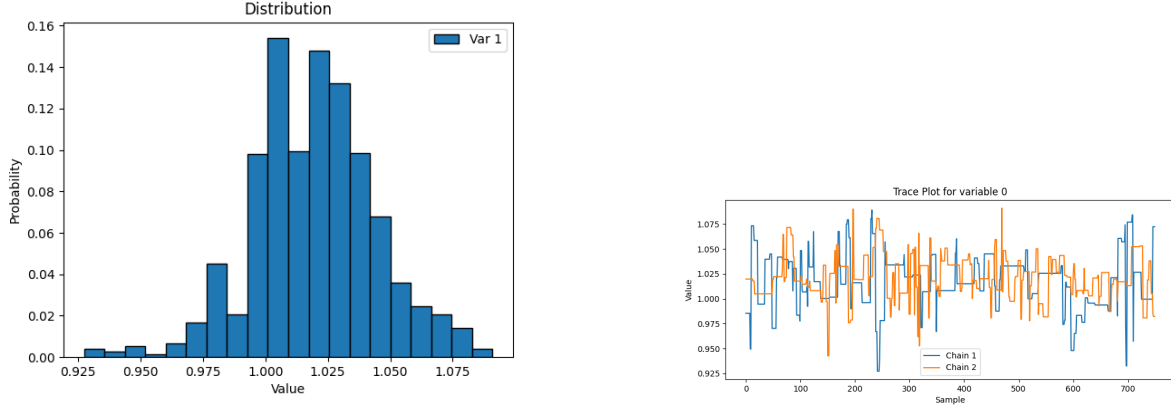


Figure 26: Trace plot and posterior distribution for 1-parameter Lotka-Volterra. Real parameter is $\delta = 21$. Two chains of length 2000 have been considered, burn-in $B = 1250$. The parameters for an optimal performance have been found to be $N_{obs} = 31$, $\sigma_{noise}^2 = 0.09649$, $\sigma_{proposal} = 0.6484$ and $s = 1$.

type structure (see Section 5.1.2 for more information), exploiting the implemented proposals to handle the individual levels. Thus, by setting the input “levels” $L > 1$ and decide an integer value of subsampling rate r_{sub} , it is then possible to use the first L models to approach the Bayesian inverse problem. The other steps (such as building a posterior and likelihood list for the model) are automated in the Library. The only drawback of this method is related to the fact that in order to have acceptable results the coarse model cannot be completely different from the fine model or the actual solution, as this would lead the two levels to diverge leaving no room for acceptance of any proposal.

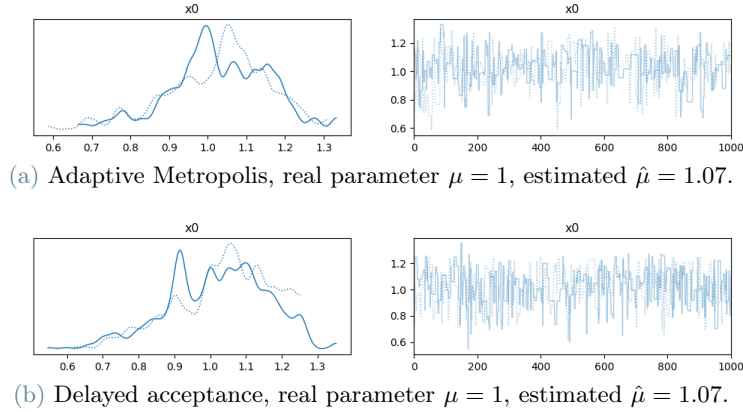


Figure 27: Estimated parameters for adapted Metropolis and delayed acceptance with adapted Metropolis sampler. In both cases, $\sigma_{noise}^2 = 0.3031$, $N_{obs} = 30$, $\sigma_{likelihood} = 3.497$, $s = 0.5$, $\sigma_{rwth}^2 = 1.955$. The subsampling rate was $r_{sub} = 2$

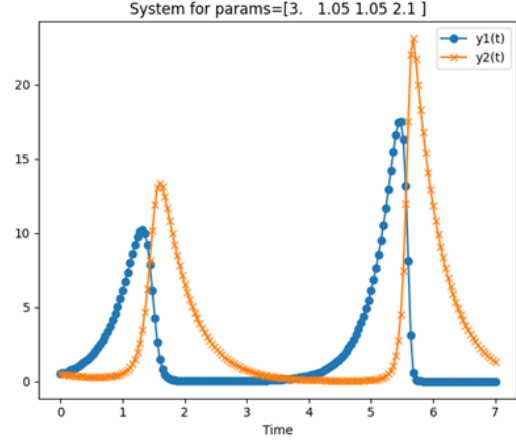
	Adapted Metropolis		DA	
	ESS bulk	ESS tail	ESS bulk	ESS tail
μ	208.0	158.0	296.0	367.0

Table 7: Comparison of ESS values between the single level and 2 levels case

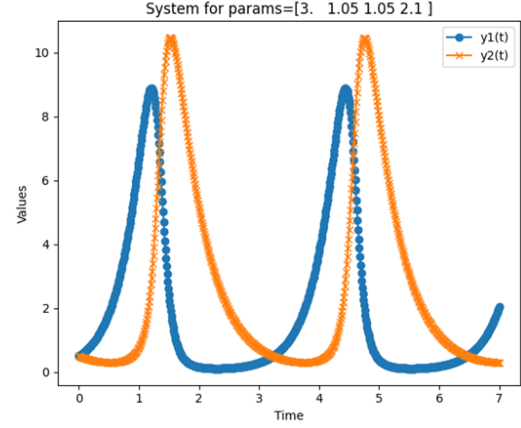
As it is possible to see, even if the expected parameter is identical in both cases, the delayed acceptance interpretation presents a better overall quality, here testified by the *ESS* value, both in bulk and tail versions. A better analysis of this fact will be given in Section 5.2.6.

5.2.5 Four parameters case

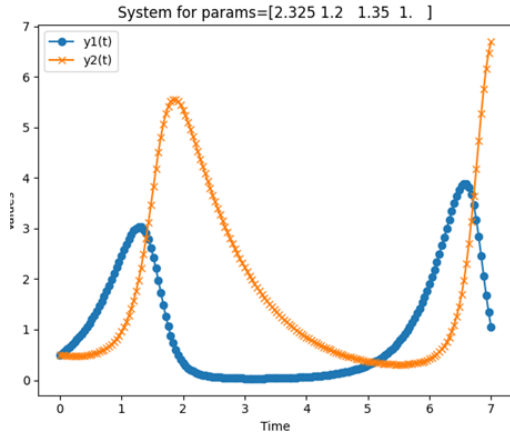
Similar to Section 5.2.3, this part considers the system of differential equations (22), defined by 4 parameters, namely a, b, c and d . It can be noticed by looking at the Figure 28 that the relationship is strongly non-linear, which therefore presents a challenge both for the construction of a forward model through a neural network, and for the subsequent parametric estimation with MCMC.



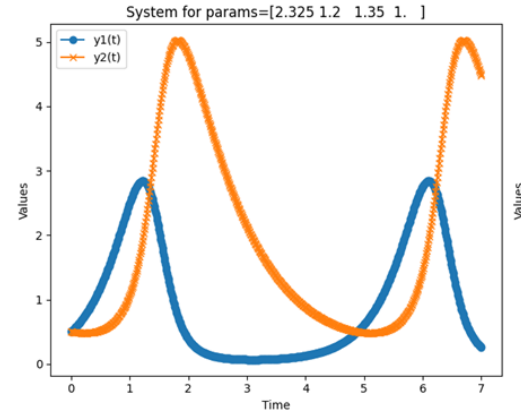
(a) Low-fidelity (Euler method) solution of system (22)



(b) High-fidelity (Runge-Kutta method) solution of system (22)



(c) Low-fidelity (Euler method) solution of system (22)



(d) High-fidelity (Runge-Kutta method) solution of system (22)

Figure 28: Euler and Runge-Kutta approximations of 4 parameters Lotka-Volterra system

With regard to the chosen model, we opted for a ‘3-step’ neural network, which could build better models for high-fidelity and low-fidelity data collections.

The data are created by solving the system with Runge-Kutta with discretization $h = 0.01$ for the high-fidelity case and forward Euler with $h = 0.04$ for the low-fidelity case. The 4 parameters are taken in the following ranges: $a \in [2.1, 3]$, $b \in [0.9, 1.5]$, $c \in [0.9, 1.5]$ and finally $d \in [1, 2.1]$. Given the many possible combinations, it was decided to consider in the training data the first 9 parameter combinations after shuffling, for the validation set the first 6 (after shuffling again) for the test set a further 16. Following an optimisation process of the hyperparameters, we obtain an overall MSE of 0.3688 and $R^2 = 0.9167$, denoting an high level of prediction. The figure 29 shows a couple of example cases of the modelling performed by the neural network, showing both the approximation performed by the first network alone and by the model as a whole.

5.2.6 Parameters a, b, c and d estimation

The need to estimate 4 parameters does not fundamentally change the operation of the code, which will simply require the number of parameters to be provided for certain characteristics, such as the mean and covariance of

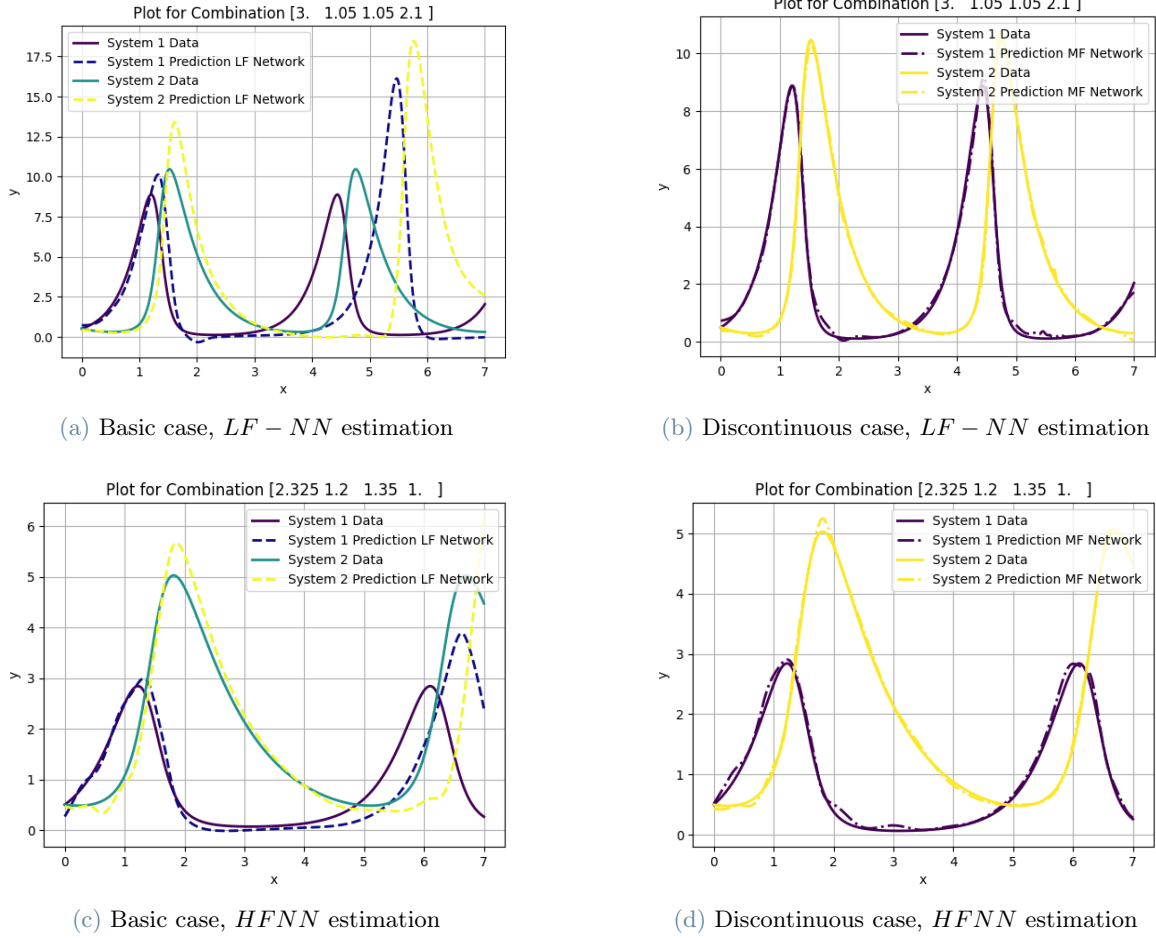


Figure 29: "3-step" NN predictions against real model 4-parameters Lotka-Volterra ODE.

the prior. The need to estimate 4 parameters does not fundamentally change the library approach. The code is designed so that the user simply has to provide some parameters in vector form (`numpy.array`) instead of scalar form, such as `mean_prior` and `cov_prior`, which are closely related to the individual parameter rather than to the algorithm itself. The additional difficulty of this type, however, lies in finding the right combination of algorithms and hyperparameters to best estimate the parameter mixture. For this reason, HPO has been performed with many combination of parameters.

HPO will seek to maximize the opposite of the sum of the relative errors of each parameter. In this paragraph, we will discuss the example of the combination with $a = 2.35$, $b = 1.2$, $c = 1.35$, $d = 1.0$. Starting from the classical Metropolis-Hastings with random walk approach, it is possible to find out that a good collection of parameters is given by $N_{obs} = 48$, $\sigma_{rw}^2 = 1.562$, $\sigma_{likelihood}^2 = 2.062$, $s = 1.033$ and $\sigma_{noise}^2 = 0.0985$. This combination, gives a rather good result, shown in Figure 30. Given this result, we would like to see if any improvement can be achieved modifying the algorithm, but the result is similar also using adaptive Metropolis. Of note is the brief analysis on a multilevel approach, exploring the behavior of MLDA. The idea is following: having approximated the relationship with a 3-step network, one already has approximations due to the (inaccurate) estimates made with the first two networks. At the point when (as in this case) the networks are still able to construct a sufficiently faithful approximation of the desired output, the relationships defined by the individual networks can be used to define the forward model then used to estimate the posterior of the given level. This approach, allows us to take full advantage of the approximations of the multi-fidelity network, including intermediate ones. It is also true, however, it could be a draw-back the fact that, even if more approximated, the intermediate networks must already be able to give a fairly convincing representation of the approximated relationship. If this does not happen, performance is likely to deteriorate quickly not making this approach any more relevant.

An initial remark can be done studying the results in Figure 31. As confirmed by theory, it turns out that the use of an adaptive type algorithm is necessary to make sure that there is appropriate exploration of the parameter space. In fact, looking at the figure, it is evident how the mere introduction of an element to adjust

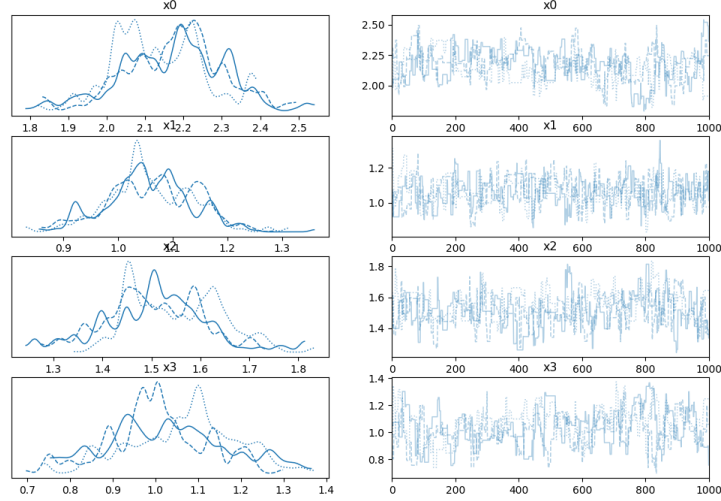
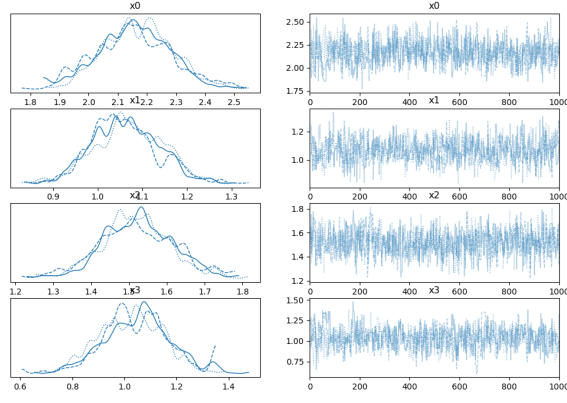
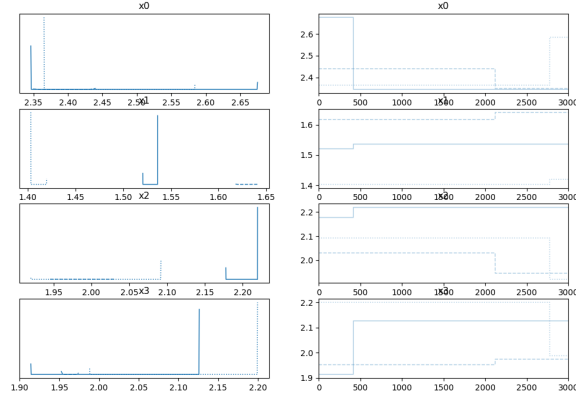


Figure 30: Posterior and trace plots of random-walk Metropolis Hastings, multiparametric case

the distribution to achieve an appropriate acceptance rate greatly improves the overall behavior. The goal now is to compare this method with the more famous MH and Adaptive metropolis to find out, if any, the advantages of this method. In the following examples we use the same parameters as in the Metropolis-Hastings random-walk case presented at the beginning of the paragraph. Regarding the quality of the method, it is useful to note how parameter estimation turns out to be similar to the traditional adaptive method. However, it should be noted what has already been observed in the paper, namely that the algorithm can have a significantly higher ESS than its single-level counterpart. The results, splitted in bulk and tail ESS (which refers respectively to the effective sample size calculated from the central part and from the tails of the posterior distribution) for an example are reported in table 8. This is particularly important since ESS represents the number of independent samples that would provide the same level of precision as the MCMC samples. More precisely a high ESS Bulk is desirable because it indicates that your MCMC algorithm is efficiently sampling from the central, most probable regions of your posterior distribution. This ensures that the central tendency (e.g., the mean, median) of the posterior is well estimated. A high ESS Tail shows that the MCMC algorithm is effectively sampling from the tails of the distribution, where extreme or rare events occur. Capturing these areas is crucial for accurately estimating quantities like credible intervals or for making predictions about rare events.



(a) Posterior distributions and trace plots 3 levels MLDA, with AM sampler



(b) Posterior distributions and trace plots 3 levels MLDA, with MH sampler

Figure 31: Examples posterior distributions and Markov chains for 3 levels MLDA with AM and MH. It is possible to notice that mixing with MH is nearly zero.

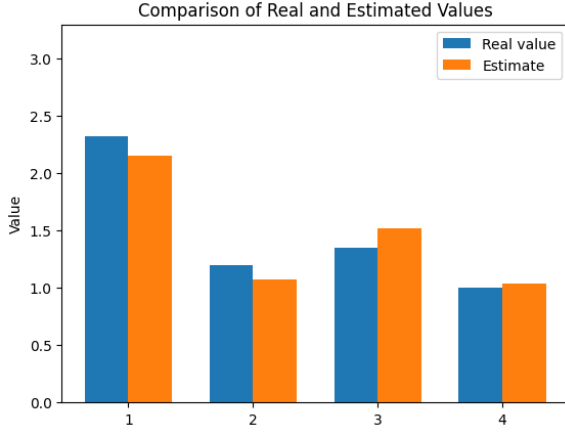
	Adapted Metropolis		MLDA	
	ESS bulk	ESS tail	ESS bulk	ESS tail
a	47	181	731	740.0
b	64	165	529.0	770.0
c	132	188	665.0	727.0
d	67	216	684.0	681.0

Table 8: Collection of ESS values

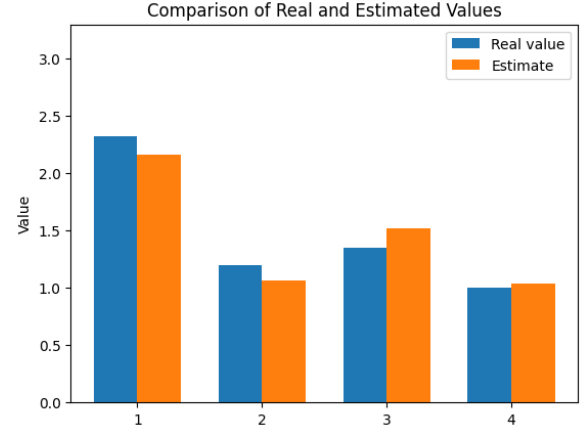
6. LSTM Networks

6.0.1 Description

Long Short-Term Memory (LSTM) networks represent a specialized architecture within recurrent neural networks (RNNs), explicitly designed to overcome the limitations of traditional RNNs, particularly the issues of vanishing and exploding gradients during backpropagation through time (BPTT). These challenges typically arise in standard RNNs when modeling long-term dependencies, causing gradients to either diminish to near zero (vanishing) or escalate exponentially (exploding) as they are propagated backward through time steps.



(a) Adapted Metropolis, real parameters $a = 2.325$, $b = 1.2$, $c = 1.35$, and $d = 1$.



(b) Multilevel delayed acceptance, real parameters $a = 2.325$, $b = 1.2$, $c = 1.35$, and $d = 1$. For this case $r_{sub} = 3$ for both levels.

Figure 32: Estimated parameters for adapted Metropolis and 3 levels multilevel delayed acceptance with adapted Metropolis sampler. It is possible to observe that both approaches can give a good approximation of the parameters.

LSTMs effectively address these problems through a sophisticated gating mechanism, allowing them to learn and retain information over extended sequences of data. This capability makes LSTMs particularly effective for tasks involving sequential data, such as time series forecasting, natural language processing (NLP), and speech recognition, where the order and temporal relationships in the data are critical [17, 62].

The central innovation of an LSTM network lies in its ability to selectively retain and update information over time, facilitated by a unique component known as the cell state. The cell state serves as a memory that is passed through the network and is modified by carefully designed structures called gates. These gates regulate the flow of information into, within, and out of the cell state, enabling the network to control which information should be remembered or forgotten at each time step. An LSTM unit consists of three essential gates:

- **Forget Gate (f_t):** The forget gate determines the portion of the previous cell state (c_{t-1}) that should be discarded. Mathematically, this gate is represented as:

$$f_t = \sigma(W_{xf} \cdot x_t + W_{hf} \cdot h_{t-1} + b_f)$$

where σ denotes the sigmoid function, W_{xf} and W_{hf} are the weight matrices corresponding to the input x_t and the previous hidden state h_{t-1} , respectively, and b_f is the bias term. The output of the forget gate is a value between 0 and 1 for each element in the cell state, where 0 signifies complete forgetting, and 1 signifies full retention of the previous cell state.

- **Input Gate (i_t):** The input gate controls the degree to which new information will be incorporated into the cell state. This process involves two steps: first, a candidate cell state $\tilde{c}_t$ is generated by applying a tanh activation function to the linear transformation of the input and the previous hidden state:

$$\tilde{c}_t = \tanh(W_{xc} \cdot x_t + W_{hc} \cdot h_{t-1} + b_c)$$

Then, the input gate calculates the amount of this candidate state to add to the current cell state:

$$i_t = \sigma(W_{xi} \cdot x_t + W_{hi} \cdot h_{t-1} + b_i)$$

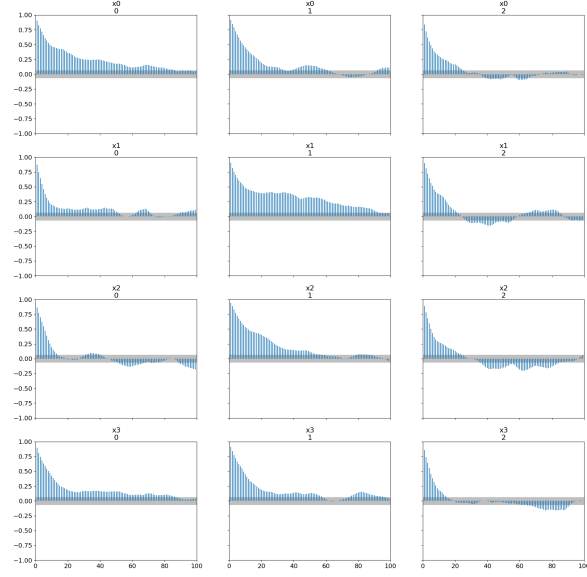
The final cell state update is then given by:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

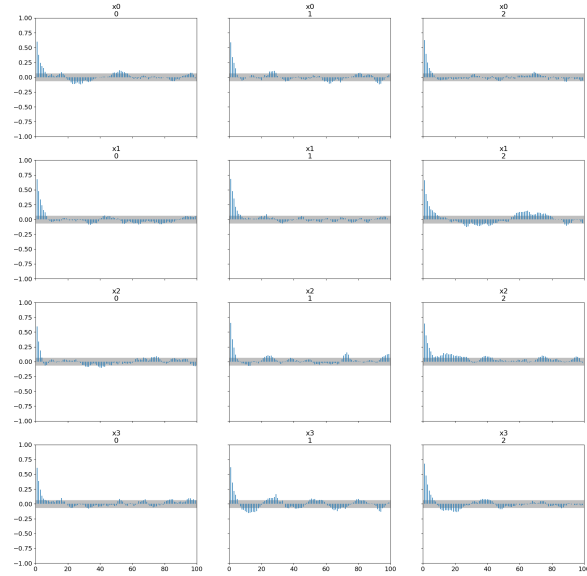
Here, $\odot$ represents element-wise multiplication.

- **Output Gate (o_t):** The output gate determines the next hidden state (h_t), which also serves as the output of the LSTM unit for the current time step. The output gate modulates the cell state to produce the hidden state using the following operation:

$$o_t = \sigma(W_{xo} \cdot x_t + W_{ho} \cdot h_{t-1} + b_o)$$



(a) Adapted Metropolis, real parameters $a = 2.325$, $b = 1.2$, $c = 1.35$, and $d = 1$.



(b) Multilevel delayed acceptance, real parameters $a = 2.325$, $b = 1.2$, $c = 1.35$, and $d = 1$.

Figure 33: Autocorrelation of the two test cases. It is possible to notice that it drops faster for MLDA. An autocovariance near to zero is a good testimony of convergence and high independence in the chains.

Finally, the hidden state is computed as:

$$h_t = o_t \odot \tanh(c_t)$$

This hidden state h_t is used as input to the next LSTM unit in the sequence and can also be utilized for tasks such as classification or prediction at the current time step.

6.0.2 Keras Implementation

When implemented using Keras, LSTMs are highly configurable, with parameters such as the number of units (which controls the dimensionality of the cell state), the activation functions, and the option to use dropout regularization to prevent overfitting (see section 2.2.1). Additionally, Keras allows for easy stacking of LSTM layers, enabling the construction of deep LSTM networks capable of capturing hierarchical representations of temporal data [13].

```
def getModel(params: dict, num_inputs: int, name: str, num_outputs: int) -> Model:
    """
    Builds and compiles an LSTM model based on the provided parameters.

    Args:
        name (str): Name of the model.
        num_inputs (int): Number of input features.
        num_outputs (int): Number of output features.
        params (dict): Dictionary containing the model parameters such as
            number of layers ('lay'), number of nodes per layer ('nodes'),
            dropout rate ('dropout'), optimizer name ('opt'), and learning
            rate ('lr').

    Returns:
        Model: A compiled Keras model.
    """
    if name == "LSTM":
        # Input layer
        inputs = Input(shape=(None, num_inputs))
        x = inputs

        # LSTM layers with Dropout
        for _ in range(params['lay']):
            x = Dropout(params['dropout'])(x)
            x = LSTM(params['nodes'], return_sequences=True)(x)

        # Dense output layer
        output = Dense(num_outputs, activation='linear')(x)

        # Create model
        model = Model(inputs=inputs, outputs=output)

        # Compile model
        optimizer = get_optimizer(params['opt'], params['lr'])
        model.compile(loss='mse', optimizer=optimizer, metrics=['mse'])

        return model

def get_optimizer(name: str, lr: float) -> tf.keras.optimizers.Optimizer:
    """
    Returns the optimizer based on the given name.

    Args:
        name (str): Name of the optimizer.
        lr (float): Learning rate for the optimizer.

    Returns:
        tf.keras.optimizers.Optimizer: The selected optimizer.

    Raises:
        ValueError: If the optimizer name is unknown.
    """
    optimizers = {
        'Adam': Adam(learning_rate=lr, amsgrad=True),
        'Nadam': Nadam(learning_rate=lr),
        'Adamax': Adamax(learning_rate=lr),
        'standardadam': 'adam'
    }
    }
```

```

if name not in optimizers:
    raise ValueError(f"Unknown optimizer name: {name}")

return optimizers[name]

```

As said before, when dealing with LSTM, the number of parameters taken into consideration during hyperparameter optimization is numerous: not only the number of nodes, the learning rate and the optimiser are considered, as before, but also the number of layers, the dropout rate and the sequence frequency f and sequence length L , which are necessary for the definition of the sliding window for LSTM. Sequence Frequency refers to the rate at which data points in a sequence are sampled over time. Mathematically, if data points x_t are sampled at intervals Δt , the frequency is defined as

$$f = \frac{1}{\Delta t}.$$

A higher frequency results in more data points within the same time span, which allows the LSTM to learn finer temporal details. However, this also introduces the potential for increased noise and higher computational complexity.

Sequence Length L represents the number of time steps that the LSTM considers in each input sequence. If a sequence starts at time t_0 and ends at $t_0 + (L - 1)\Delta t$, then L is the number of past observations used for making predictions. Formally, the input to the LSTM can be represented as a matrix of size $L \times n$, where n is the number of features per time step. A longer sequence length enables the LSTM to capture long-term dependencies, but it also increases the risk of overfitting and computational cost.

During the training phase, the Sliding Window technique is employed to generate multiple training samples from a time series. This means creating overlapping sub-sequences of length L that move over the entire sequence. For a time series of length T , the sliding window generates $T - L + 1$ training samples. Each sample consists of an input sequence $[x_t, x_{t+1}, \dots, x_{t+L-1}]$ and its corresponding target output x_{t+L} . This method is used for training LSTM models on time-dependent data, as it provides sufficient training data while preserving temporal dependencies. In the code, it is implemented in the *LSTM_network* class.

6.1. Example IV: Burger's equation

Let us consider the time-dependent solution to the 1 dimensional Burgers' equation:

$$u_{HF}(x, t, Re) = \frac{x}{(t+1) \left(1 + \exp \left(\frac{Re}{4t+4} \right) \left(\frac{t+1}{A_0} \right)^{0.5} \right)}, \quad (23)$$

where $u_{HF}(x, t, Re)$ represents a velocity field, x is the spatial coordinate, t the time variable, Re the Reynolds number and

$$A_0 = \exp \left(\frac{Re}{8.0} \right). \quad (24)$$

Given a low-fidelity version of this solution, which is:

$$u_{LF}(x, t, Re) = \frac{x}{(t+1) \left(1 + \exp \left(\frac{Re \cdot x}{4t+4} \right) \left(\frac{t+1}{A_0} \right)^{0.5} \right)} \quad (25)$$

we would like to build an architecture able to start from the approximate solution (25) and reach the real solution (23). The LSTM architecture is chosen due to its ability to handle sequential data and capture temporal dependencies, making it suitable for time series prediction tasks. More specifically, we develop a LSTM neural network as a surrogate model to map low-fidelity (LF) to high-fidelity (HF) POD coefficients. The goal is to predict the HF coefficients from the LF coefficients for each time instance t_k by minimizing the mean squared error between the true and predicted HF coefficients. The LSTM network, denoted by is trained on pairs of LF and HF POD coefficients, and its network parameters θ_{LSTM} are optimized using the Adam algorithm.

As already mentioned, in this study we use a Reduced-Order Model (ROM) to address the computational challenges associated with HF simulations. The ROM is constructed using Proper Orthogonal Decomposition (POD), a technique that reduces the dimensionality of complex systems while preserving their essential dynamics. First of all, a matrix is generated from HF solutions obtained over a limited set of parameter configurations. This snapshot matrix X_{HF} contains high-dimensional data that represents the key features of the system's behavior.

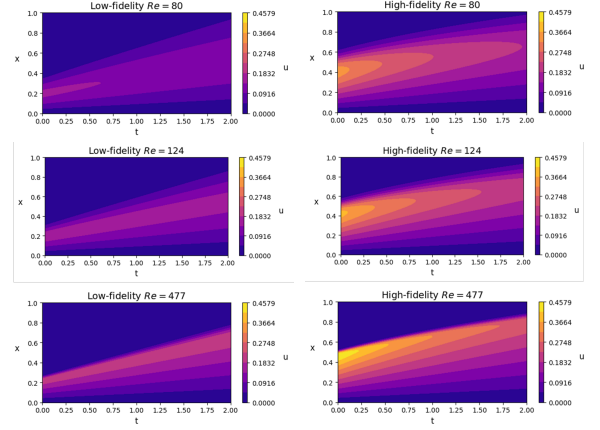
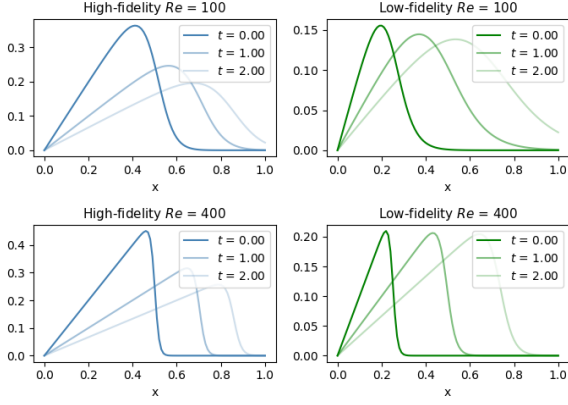


Figure 34: Comparison of low-fidelity and high-fidelity solution for different Reynolds number

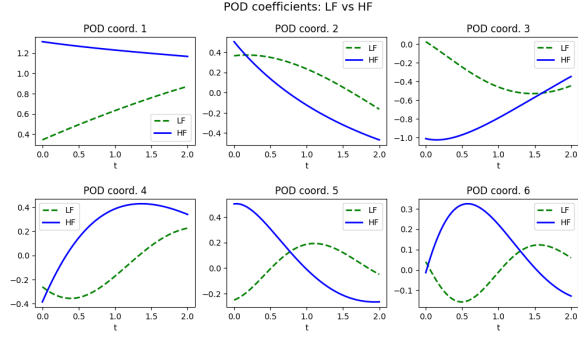
To reduce the dimensionality, the snapshot matrix X_{HF} is subject to Singular Value Decomposition (SVD), which decomposes it into three matrices: $X_{\text{HF}} = U\Sigma V^T$. Here, U contains the left singular vectors (spatial modes), Σ is a diagonal matrix of singular values, and V^T contains the right singular vectors (temporal modes). The singular values in Σ quantify the importance of each corresponding mode, and by retaining only the most significant modes — those associated with the largest singular values — we obtain a reduced set of orthonormal basis functions.

The reduced basis is defined by selecting the first N_{RB} columns of U , where N_{RB} is chosen based on a predefined tolerance ε_{POD} that ensures the retained modes capture a substantial portion of the original system’s variance. This reduced basis allows for the projection of the HF data into a lower-dimensional subspace, significantly simplifying the problem while maintaining its critical features.

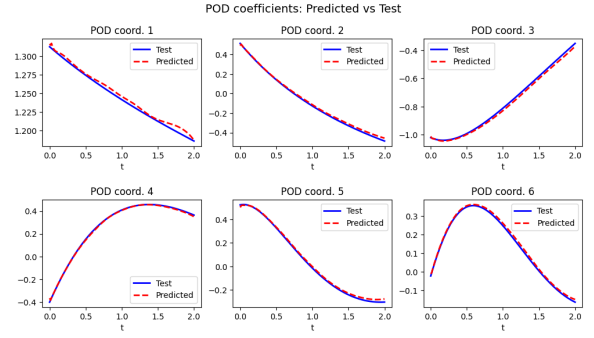
The ROM then transforms the original high-dimensional problem into this reduced space, where the solution is represented by a smaller set of coefficients corresponding to the reduced basis. This projection not only reduces the computational burden but also facilitates faster simulations for different parameter instances, making the ROM an efficient surrogate model. By retaining the most important modes, the ROM achieves a balance between computational efficiency and accuracy, providing a powerful tool for scenarios that require numerous simulations across varying parameters. To apply the MF-POD method, we perform the POD reduction on the HF snapshots considering 16 modes. LF data are lifted to the HF spatial dimensionality and then, for both the levels, POD coefficients are computed by projecting the data onto the reduced basis and then fed to the LSTM neural network for training.

6.1.1 LSTM results

After a tuning phase through HPO, it was possible to find a combination of parameters and number of layers that would allow the architecture to show its full potential. Looking at Figure 36, it can be seen that from a qualitative point of view, the approximation carried out by the model is basically perfect. As further proof of this, it is possible to arrive at the same observation by studying the comparison between prediction and HF solution at fixed Reynolds number in Figure 36.

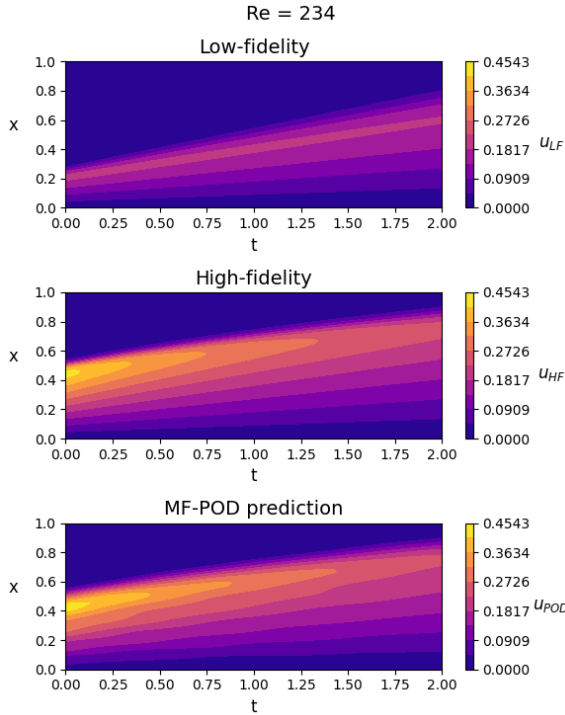


(a) Visual comparison of low and high-fidelity coefficients

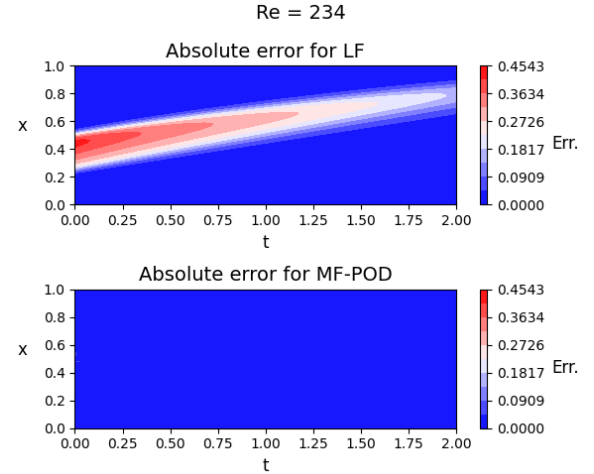


(b) Predicted coefficients

Figure 35: POD coefficients: it is possible to notice that the LSTM is able to almost perfectly approximates the HF elements starting from the LF given as an input.



(a) Comparison of estimate with different fidelity levels



(b) Absolute error

Figure 36: At fixed Re , the approximation is nearly perfect

6.1.2 Parameter Re estimation

The next step is to estimate the Reynolds number Re through MCMC. This is particularly complex compared to the previous cases, as the network, which acts as a forward model, not only has as input the parameters to be estimated and the variable, usually time, but also has the 16 reductions of the u_{LF} computed following the relation (25), obtained as explained in Section 6.1. The parameter estimated at a given step is then associated with the observation points on the time domain and the values of u_{LF}^{POD} achieved using the same 16 modes obtained during data creation for network training. From a code point of view, this involves redefining by overriding the function responsible for binding the estimated parameter to the observation points on the time domain. In this case, the overriding basically consists of providing a function associated with the class associated to the specific problem, which allows each step to introduce this additional calculation to fully define the input. This type of very generic overriding, included in the `LSTM_network` class, allows its reuse in any network case of this type, where always, in addition to the physical domain input and the parametric input there must be an input related to the ROM approximation. A further example will be that in section 6.2. More precisely, the overriding is on functions `param_inverse` and `inverse_cuqi`, devoted to the creation of the proper input for

the evaluation in MCMC. A result as an example is in Figure 37

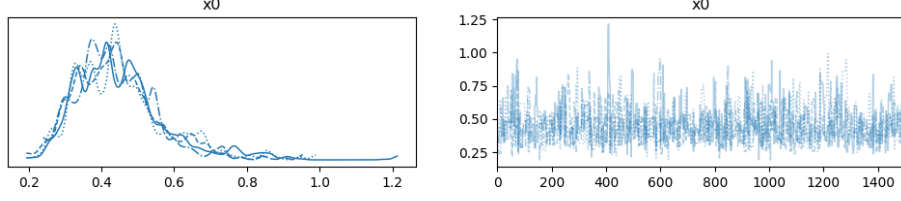


Figure 37: Trace plot of a Random Walk Metropolis-Hastings MCMC experiment with 4 chains of length 3000, burn-in=2000. The estimated value is 0.448, the real normalized $Re_{norm} = 0.368$

6.2. Example V: Reaction-Diffusion problem

Following the example I in [16], and following a procedure similar to the one described in Section 4.4, we create a dataset from PDE (11) to recreate the LSTM structure there defined.

Analogously to section 6.1, we introduce a POD reduction to reduce the dimensionality of the problem, considering only the 9 more meaningful modes. Those, together with the time coordinate $t \in [0, 40]$ and parameter μ define the input for the LSTM network. The expected output is the HF definition of the velocity field.

As well as in Section 6.1.2, the problem arises of making it possible to evaluate the LF function at each step. In this case, however, one does not have an explicit function to evaluate. Consequently, the idea is to construct LSTMs that act as surrogate models of the 9 necessary components, for each $t \in [0, 40]$ and for each $\mu \in [0.5, 1.5]$. Example results are shown in the Figure 38. Given the time required to create the 9 models, for ease and cleanliness of the code these are saved in the `fwd_models` folder, and loaded only when needed.

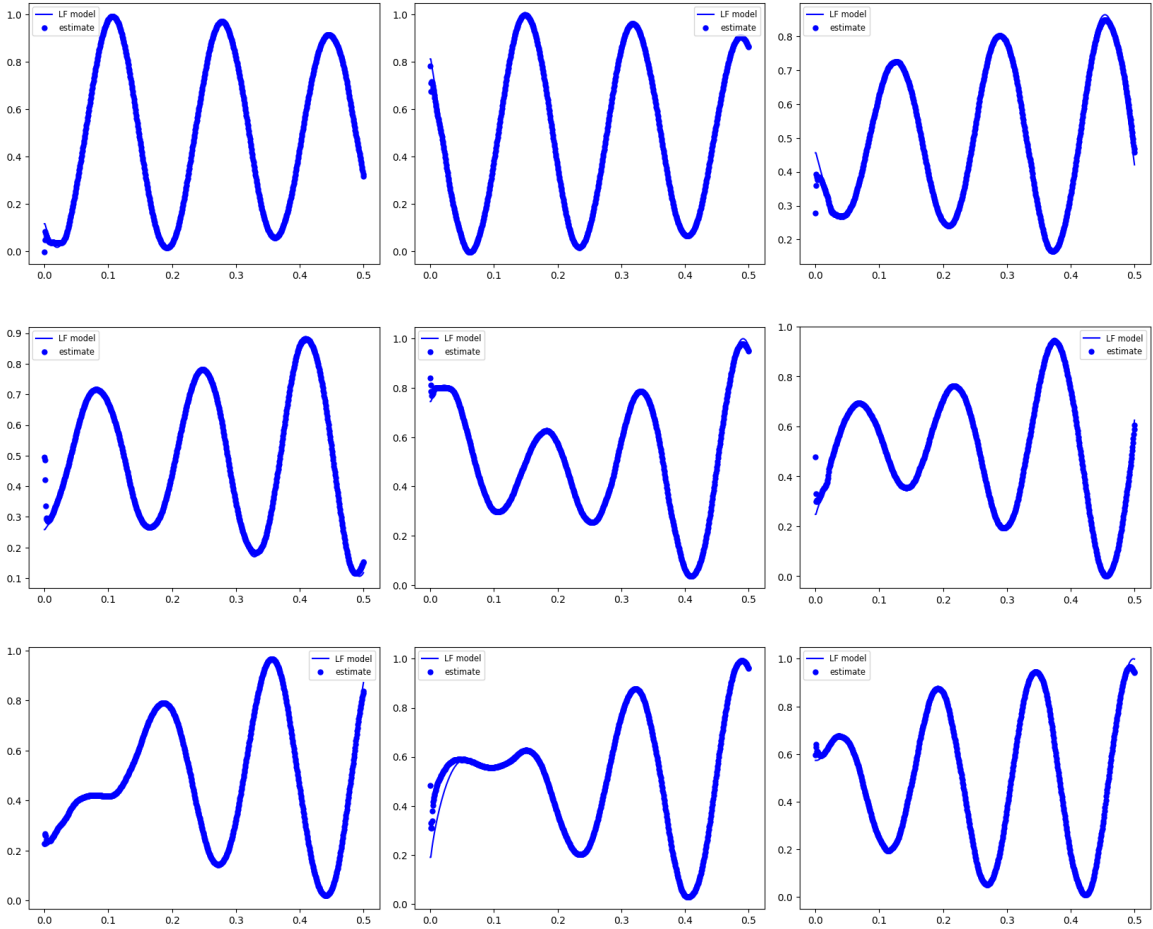


Figure 38: Collection of the 9 normalized POD coefficients approximated through LSTM architectures with respect to time, $\mu = 0.5$. The input t and the output u_{LF}^{POD} in the plot are normalized.

In this case, the computational weight is higher than before, due to both the dimensionality and complexity

of the problem. However, it is possible to obtain a fairly accurate estimate of the initial parameters, which is exemplified here in the result in Figure 39.

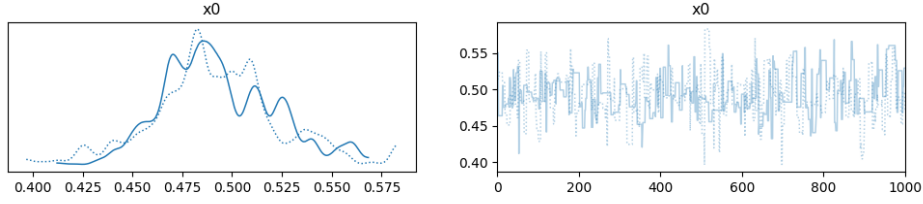


Figure 39: Trace plot of a Metropolis-Hastings MCMC experiment with 2 chains of length 3000, burn-in=2000. The estimated value is 0.492, the real $\mu = 0.5$. To obtain the result, after an HPO, it has been decided to use $\sigma_{noise}^2 = 0.001$, scaling $s = 1.5443$, $\sigma_{likelihood} = 0.7$, $\sigma_{proposal}^2 = 0.96$, $N_{obs} = 150$.

7. Conclusions

During the research work, many aspects relating to the constitution of optimal surrogate models were investigated, starting with the definition of multifidelity type architectures, which give the possibility of optimising the performance of neural networks against an improvement in the cost required to define a dataset, and ending with the study of methods for solving backward problems with the contribution of multilevel structures. From an implementation point of view, the construction of a general structure capable of handling as many cases as possible makes it possible to investigate many other aspects relating to cases similar to those already established, such as ODEs, but also to more modern cases, such as the ROM approach applied to PDEs. The test cases given as examples not only seem to validate the general functionality of the model, but also allow us to see and study innovative cases, such as the joint exploitation of multilevel approaches for both forward and backward surrogate models.

References

- [1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 2623–2631, 2019.
- [2] Amal MA Alghamdi, Nicolai AB Riis, Babak M Afkham, Felipe Uribe, Silja L Christensen, Per Christian Hansen, and Jakob S Jørgensen. Cuqipy: Ii. computational uncertainty quantification for pde-based inverse problems in python. *Inverse Problems*, 40(4):045010, 2024.
- [3] Peshawa Jamal Muhammad Ali, Rezhna Hassan Faraj, Erbil Koya, Peshawa J Muhammad Ali, and Rezhna H Faraj. Data normalization and standardization: a technical report. *Mach Learn Tech Rep*, 1(1):1–6, 2014.
- [4] Nurshazlyn Mohd Aszemi and PDD Dominic. Hyperparameter optimization in convolutional neural network using genetic algorithms. *International Journal of Advanced Computer Science and Applications*, 10(6), 2019.
- [5] Nathan Baker, Frank Alexander, Timo Bremer, Aric Hagberg, Yannis Kevrekidis, Habib Najm, Manish Parashar, Abani Patra, James Sethian, Stefan M. Wild, and Karen Willcox. Workshop report on basic research needs for scientific machine learning: Core technologies for artificial intelligence. Report, U.S. Department of Energy, ASCR, 2019.
- [6] Utpal Banerjee, Rudolf Eigenmann, Alexandru Nicolau, and David A Padua. Automatic program parallelization. *Proceedings of the IEEE*, 81(2):211–243, 1993.
- [7] Peter Belcák and Roger Wattenhofer. Periodic extrapolative generalisation in neural networks, 2022.
- [8] Michael Betancourt. A conceptual introduction to hamiltonian monte carlo. *arXiv preprint arXiv:1701.02434*, 2017.
- [9] Arjun Nitin Bhagoji, Daniel Cullina, Chawin Sitawarin, and Prateek Mittal. Enhancing robustness of machine learning systems via data transformations. In *2018 52nd Annual Conference on Information Sciences and Systems (CISS)*, pages 1–5. IEEE, 2018.

- [10] Diana Cai and Ryan P Adams. Multi-fidelity monte carlo: a pseudo-marginal approach. Advances in Neural Information Processing Systems, 35:21654–21667, 2022.
- [11] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In International conference on machine learning, pages 1597–1607. PMLR, 2020.
- [12] Siddhartha Chib and Edward Greenberg. Understanding the metropolis-hastings algorithm. The american statistician, 49(4):327–335, 1995.
- [13] Francois Chollet et al. Building autoencoders in keras. The Keras Blog, 14, 2016.
- [14] J. Christen and Colin Fox. Markov chain monte carlo using an approximation. Journal of Computational and Graphical Statistics - J COMPUT GRAPH STAT, 14:795–810, 12 2005.
- [15] Paolo Conti, Giorgio Gobat, Stefania Fresca, Andrea Manzoni, and Attilio Frangi. Reduced order modeling of parametrized systems through autoencoders and sindy approach: continuation of periodic solutions. Computer Methods in Applied Mechanics and Engineering, 411:116072, 2023.
- [16] Paolo Conti, Mengwu Guo, Andrea Manzoni, Attilio Frangi, Steven L. Brunton, and J. Nathan Kutz. Multi-fidelity reduced-order surrogate modelling. Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences, 480(20230655), 2024.
- [17] Paolo Conti, Mengwu Guo, Andrea Manzoni, and Jan S. Hesthaven. Multi-fidelity surrogate modeling using long short-term memory networks. Computer Methods in Applied Mechanics and Engineering, 404:115811, 2023.
- [18] Corinna Cortes, Mehryar Mohri, and Afshin Rostamizadeh. L2 regularization for learning kernels, 2012.
- [19] George E Dahl, Tara N Sainath, and Geoffrey E Hinton. Improving deep neural networks for lvcsr using rectified linear units and dropout. In 2013 IEEE international conference on acoustics, speech and signal processing, pages 8609–8613. IEEE, 2013.
- [20] Gonzalo I Diaz, Achille Fokoue-Nkoutche, Giacomo Nannicini, and Horst Samulowitz. An effective algorithm for hyperparameter optimization of neural networks. IBM Journal of Research and Development, 61(4/5):9–1, 2017.
- [21] Tim J Dodwell, Christian Ketelsen, Robert Scheichl, and Aretha L Teckentrup. A hierarchical multilevel markov chain monte carlo algorithm with applications to uncertainty quantification in subsurface flow. SIAM/ASA Journal on Uncertainty Quantification, 3(1):1075–1108, 2015.
- [22] Tim J Dodwell, Christian Ketelsen, Robert Scheichl, and Aretha L Teckentrup. Multilevel markov chain monte carlo. Siam Review, 61(3):509–545, 2019.
- [23] Peter I Frazier. A tutorial on bayesian optimization. arXiv preprint arXiv:1807.02811, 2018.
- [24] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In Yee Whye Teh and Mike Titterton, editors, Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics, volume 9 of Proceedings of Machine Learning Research, pages 249–256, Chia Laguna Resort, Sardinia, Italy, 13–15 May 2010. PMLR.
- [25] Antonio Gulli and Sujit Pal. Deep learning with Keras. Packt Publishing Ltd, 2017.
- [26] Mengwu Guo, Andrea Manzoni, Maurice Amendt, Paolo Conti, and Jan S. Hesthaven. Multi-fidelity regression using artificial neural networks: Efficient approximation of parameter-dependent output quantities. Computer Methods in Applied Mechanics and Engineering, 389:114378, 2022.
- [27] Heikki Haario, Eero Saksman, and Johanna Tamminen. Adaptive proposal distribution for random walk metropolis algorithm. Computational statistics, 14:375–395, 1999.
- [28] Heikki Haario, Eero Saksman, and Johanna Tamminen. An adaptive Metropolis algorithm. Bernoulli, 7(2):223 – 242, 2001.
- [29] Matthew D Hoffman, Andrew Gelman, et al. The no-u-turn sampler: adaptively setting path lengths in hamiltonian monte carlo. J. Mach. Learn. Res., 15(1):1593–1623, 2014.

- [30] Jérôme Idier. Bayesian approach to inverse problems. John Wiley & Sons, 2013.
- [31] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017.
- [32] Yim-Kul Lee and William T Rhodes. Nonlinear image processing by a rotating kernel transformation. Optics letters, 15(23):1383–1385, 1990.
- [33] Petro Liashchynskiy and Pavlo Liashchynskiy. Grid search, random search, genetic algorithm: a big comparison for nas. arXiv preprint arXiv:1912.06059, 2019.
- [34] Dehao Liu and Yan Wang. Multi-fidelity physics-constrained neural network and its application in materials modeling. Journal of Mechanical Design, 141(12):121403, 2019.
- [35] Ricardo Llugsi, Samira El Yacoubi, Allyx Fontaine, and Pablo Lupera. Comparison between adam, adamax and adam w optimizers to implement a weather forecast based on neural networks for the andean city of quito. In 2021 IEEE Fifth Ecuador Technical Chapters Meeting (ETCM), pages 1–6. IEEE, 2021.
- [36] Juan P Madrigal-Cianci, Fabio Nobile, and Raul Tempone. Analysis of a class of multilevel markov chain monte carlo algorithms based on independent metropolis–hastings. SIAM/ASA Journal on Uncertainty Quantification, 11(1):91–138, 2023.
- [37] Kiran Maharana, Surajit Mondal, and Bhushankumar Nemade. A review: Data pre-processing and data augmentation techniques. Global Transitions Proceedings, 3(1):91–99, 2022.
- [38] Navin Kumar Manaswi and Navin Kumar Manaswi. Understanding and working with keras. Deep learning with applications using Python: Chatbots and face, object, and speech recognition with TensorFlow and Keras, pages 31–43, 2018.
- [39] Xuhui Meng, Hessam Babaei, and George Em Karniadakis. Multi-fidelity bayesian neural networks: Algorithms and applications. Journal of Computational Physics, 438:110361, 2021.
- [40] Xuhui Meng and George Em Karniadakis. A composite neural network that learns from multi-fidelity data: Application to function approximation and inverse pde problems. Journal of Computational Physics, 401:109020, 2020.
- [41] Jojo Moolayil and Jojo Moolayil. An introduction to deep learning and keras. Learn keras for deep neural networks: A fast-track approach to modern deep learning with Python, pages 1–16, 2019.
- [42] Francisco J Moreno-Barea, Fiammetta Strazzer, José M Jerez, Daniel Urda, and Leonardo Franco. Forward noise adjustment scheme for data augmentation. In 2018 IEEE symposium series on computational intelligence (SSCI), pages 728–734. IEEE, 2018.
- [43] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I Jordan, et al. Ray: A distributed framework for emerging {AI} applications. In 13th USENIX symposium on operating systems design and implementation (OSDI 18), pages 561–577, 2018.
- [44] Mohammad Motamed. A multi-fidelity neural network surrogate sampling method for uncertainty quantification. International Journal for Uncertainty Quantification, 10(4), 2020.
- [45] Alfio Quarteroni, Riccardo Sacco, and Fausto Saleri. Numerical mathematics, volume 37. Springer Science & Business Media, 2006.
- [46] Nicolai AB Riis, Amal MA Alghamdi, Felipe Uribe, Silja L Christensen, Babak M Afkham, Per Christian Hansen, and Jakob S Jørgensen. Cuqipy: I. computational uncertainty quantification for inverse problems in python. Inverse Problems, 40(4):045009, 2024.
- [47] Christian Robert and George Casella. A short history of markov chain monte carlo: Subjective recollections from incomplete data. Statistical Science - STAT SCI, 26, 08 2008.
- [48] Gareth O. Roberts and Jeffrey S. Rosenthal. General state space markov chains and mcmc algorithms. Probability Surveys, 1(none), January 2004.
- [49] Daniel Rudolf and Björn Sprungk. On a generalization of the preconditioned crank–nicolson metropolis algorithm. Foundations of Computational Mathematics, 18:309–343, 2018.

- [50] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. Nature, 323:533–536, 1986.
- [51] Linus Seelinger, Anne Reinartz, Mikkel B Lykkegaard, Amal Mohammed A Alghamdi, David Aristoff, Wolfgang Bangerth, Jean Bénézech, Matteo Diez, Kurt Frey, John D Jakeman, et al. Democratizing uncertainty quantification. arXiv preprint arXiv:2402.13768, 2024.
- [52] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. The journal of machine learning research, 15(1):1929–1958, 2014.
- [53] Andrew M Stuart. Inverse problems: a bayesian perspective. Acta numerica, 19:451–559, 2010.
- [54] Matteo Torzoni, Andrea Manzoni, and Stefano Mariani. A multi-fidelity surrogate model for structural health monitoring exploiting model order reduction and artificial neural networks. Mechanical Systems and Signal Processing, 197:110376, 2023.
- [55] Lloyd N Trefethen. Spectral methods in MATLAB. SIAM, 2000.
- [56] R.E. Uhrig. Introduction to artificial neural networks. In Proceedings of IECON '95 - 21st Annual Conference on IEEE Industrial Electronics, volume 1, pages 33–37 vol.1, 1995.
- [57] H Wang, ZeZheLei, X Zhang, B Zhou, and J Peng. Machine learning basics. Deep learning, pages 98–164, 2016.
- [58] Peter J Wangersky. Lotka-volterra population models. Annual Review of Ecology and Systematics, 9:189–218, 1978.
- [59] Shuhei Watanabe. Tree-structured parzen estimator: Understanding its algorithm components and their roles for better empirical performance. arXiv preprint arXiv:2304.11127, 2023.
- [60] Lichao Wu, Guilherme Perin, and Stjepan Picek. I choose you: Automated hyperparameter tuning for deep learning-based side-channel analysis. IEEE Transactions on Emerging Topics in Computing, 2022.
- [61] Xue Ying. An overview of overfitting and its solutions. In Journal of physics: Conference series, volume 1168, page 022022. IOP Publishing, 2019.
- [62] Yong Yu, Xiaosheng Si, Changhua Hu, and Jianxun Zhang. A review of recurrent neural networks: Lstm cells and network architectures. Neural computation, 31(7):1235–1270, 2019.
- [63] Leila Zahedi, Farid Ghareh Mohammadi, Shabnam Rezapour, Matthew W Ohland, and M Hadi Amini. Search algorithms for automated hyper-parameter tuning. arXiv preprint arXiv:2104.14677, 2021.
- [64] Liu Ziyin, Tilman Hartwig, and Masahito Ueda. Neural networks fail to learn periodic functions and how to fix it. Advances in Neural Information Processing Systems, 33:1583–1594, 2020.